A Comparative Analysis of Relational Database Access-Layer Performance in Express.js for PostgreSQL and MySQL

Mateusz Miotk[a,*]

[a]*Faculty of Mathematics, Physics and Informatics, University of Gdańsk, Wita Stwosza 57, 80-308, Gdańsk, Poland*

Abstract

Context: Node.js/Express services reach relational databases through a spectrum of access layers (native drivers, query builders, and object-relational mappers, or ORMs), yet practitioners choose between them largely on vendor benchmarks that are fragmented, PostgreSQL-only, and rarely report the tail (p99). **Objective:** We quantify, reproducibly and independently of the evaluated vendors, the access layer's performance cost across layer, engine, and access pattern, and decompose it into SQL execution versus mapping strategy. **Methods:** An identical Express application is served through nine idiomatic access layers (two native drivers, a query builder, and six ORMs) and two tuned native baselines, over five access patterns, against PostgreSQL and MySQL. We report throughput and tail latency (p99) jointly from 25 independent replicates per cell; an automated cross-check verifies byte-identical responses across layers before measurement, and inserts run under the engines' default durability from a physically rebuilt state. **Results:** Abstraction cost concentrates in relation-materializing patterns: the deep/nested fetch spreads 6.5× (PostgreSQL) and 4.5× (MySQL) between the native driver and the heaviest data-mapper ORM, growing with the size of the fetched graph. A same-SQL control collapses the spread to at most 2.2×: most of the deficit lies upstream of raw SQL execution, in each

*Corresponding author.
Email address: mateusz.miotk@ug.edu.pl (Mateusz Miotk)

layer's query-generation and result-construction path (its default eager-loading strategy, query count, and object hydration). Read rankings transfer across engines (Spearman $\rho \geq 0.89$); the write ranking does not ($\rho = 0.43$): under default durability, inserts are engine-bound on MySQL yet layer-dependent on PostgreSQL. Prisma matches or leads native throughput on several patterns at roughly five times the CPU of every other layer. **Conclusion:** The choice of access layer matters most where the layer must assemble a nested object graph; a reproducible harness and a checklist of benchmarking pitfalls specific to this genre are released as a replication package.

Keywords: database access layer, object-relational mapping, Node.js, performance benchmarking, tail latency, empirical software engineering

---

1. Introduction

Express.js remains the most widely used web framework for Node.js, and almost every non-trivial service built on it eventually reaches a relational database through some *access layer*. The choice is not settled by the language or the framework: access layers trade raw performance for ergonomics, compile-time type safety, and protection from pitfalls such as the N+1 query problem, where a naively written nested fetch silently issues one query per parent row instead of one query for the whole result graph [1], a pervasive anti-pattern in real-world database-backed applications (Section 2). Native drivers expose the wire protocol directly (`pg` for PostgreSQL, `mysql2` for MySQL); query builders assemble SQL programmatically (Knex); and object-relational mappers (ORMs) map rows to objects and manage relationships automatically, following patterns such as Data Mapper and Active Record [2] (the lightweight ORM Drizzle, together with Prisma, Sequelize, TypeORM, Objection, and MikroORM). Node.js backend performance under load is itself a recognized empirical topic [3, 4, 5], but that literature treats the runtime and web framework as the unit of comparison rather than varying the database access layer.

In practice, the most visible performance comparisons are *vendor bench-*

marks: plentiful but narrow, published by the maintainers of the compared products, confined almost exclusively to PostgreSQL, and settling on a single metric family rather than reporting throughput and the tail together (Section 2 reviews these in detail). None reports the p99 behavior that dominates perceived performance under load [6, 7]. The peer-reviewed literature is sparser still, limited to at most three ORMs, confined to PostgreSQL, and, where it benchmarks PostgreSQL against MySQL directly, holding the access layer fixed rather than varying it [8, 9, 10, 11, 12] (Section 2 surveys each).

Taken together, this leaves a gap along four axes: (1) no access-layer comparison includes MySQL, and no PostgreSQL/MySQL benchmark varies the access layer; (2) no vendor-independent study spans the taxonomy broadly, since the broad-coverage comparisons are vendor-authored and academic work covers at most three libraries [10, 11]; (3) throughput and the p99 tail are never reported jointly, the sole vendor suite that pairs throughput with a percentile stopping at a single P95 at one load point [13]; and (4) no access-layer comparison measures client-observed performance through a production HTTP framework: academic studies instrument query execution inside the application process [11], and the Express-based work that does measure the request round trip compares runtime optimizations on a single stack rather than access layers [14].

This paper addresses that gap with a single, vendor-independent benchmark: we serve an *identical* Express application through all nine access layers (two native drivers, the Knex query builder, and six ORMs: Drizzle, Prisma, Sequelize, TypeORM, Objection, and MikroORM) over five access patterns chosen to span common CRUD operations, not sampled from a measured production workload: a point read by primary key, a keyset range scan over the most recent rows, a deep/nested fetch materializing a post with its author and every comment (each with its own comment-author), a per-author aggregation of post counts and view sums, and an insert. Seven of the nine layers (all but the two engine-specific native drivers) run against *both* engines under an identical schema, dataset, and pool size; two tuned native-driver baselines (prepared-statement variants of `pg` and `mysql2`) bound what hand-tuned driver use achieves. Every (layer,

engine, pattern) combination reports throughput (req/s) and tail latency (p99) jointly, from 25 independent replicates. The study separates two estimands: the *default-configuration* effect of each layer used idiomatically (RQ1–RQ3), and a controlled *same-SQL* effect isolating each layer's execution machinery from its query-generation and hydration strategy. The full harness (adapters, seed data, an autocannon-driven [15] load runner, and an automated cross-check that all layers return *byte-identical* responses on the four non-mutating patterns) is released as an open replication package.[1]

Three research questions structure the study:

RQ1 *(overhead):* How large is the throughput and tail-latency overhead of each access layer relative to the native-driver baseline, and how does it vary across the five access patterns?

RQ2 *(engine dependence):* Is the relative ordering of access layers consistent across PostgreSQL and MySQL, or is it engine-dependent?

RQ3 *(consequential patterns):* For which access patterns is the choice of access layer most consequential?

This paper makes four contributions. The first two are durable; the third is a version-dated snapshot of a fast-moving ecosystem, whereas the design and harness outlast the pinned versions:

- An **open, reproducible harness**: a uniform adapter contract, a deterministic seed, a matrix runner with independent replicates and paired request streams, a physical write-state rebuild, a same-SQL control with server-side protocol evidence, a coordinated-omission-corrected open-loop generator, and an automated **correctness cross-check** asserting **byte-identical** responses across layers on the four non-mutating patterns (Section 3).

[1] Harness, deterministic seed, all adapters, raw per-cell measurements, and the table/figure-generating scripts: `https://github.com/mmiotk/express-db-access-performance`; release v1.2.0 is permanently archived at Zenodo, `https://doi.org/10.5281/zenodo.21356266`.

- A **vendor-independent, broad-coverage benchmark design** spanning the taxonomy's three tiers (native driver → query builder → six ORMs), **dual-engine** (PostgreSQL and MySQL), measured at the **Express level**, reporting throughput and tail latency **jointly**, the combination missing from prior art.

- Empirical findings on the pinned versions: two not obvious in advance, that a **query-engine ORM can tie, and on several patterns lead, the native driver** at a substantial application-CPU premium (quantified by an equal-CPU control, Section 4), and that **whether writes are layer-bound depends on the engine** (visible only in a dual-engine design); and a controlled **confirmation and quantification** of the expected effects: abstraction cost concentrates in relation-materializing patterns and grows with the fetched graph, and the coarse layer ordering of reads is stable across engines.

- A practitioner-oriented **checklist of pitfalls specific to access-layer benchmarking** (aggregation fan-out, OFFSET versus keyset pagination, write-workload contamination, and query-formulation confounds), surfaced during development, several caught by the correctness cross-check and the rest by performance analysis.

The remainder of the paper surveys related work in more detail (Section 2), details the benchmark design (Section 3), reports results per research question (Section 4), discusses their implications (Section 5) alongside threats to validity (Section 6), and concludes (Section 7).

2. Related Work

We organize prior work along four axes on which our study differs from it (taxonomy coverage, engine coverage, whether throughput and tail latency are reported jointly, and vendor independence), summarized in Table 1. We located this work by searching the ACM Digital Library, IEEE Xplore, Scopus, and Google Scholar for combinations of *ORM*, *query builder*, *database access*

layer, *Node.js*, *Express*, *PostgreSQL*, and *MySQL* with *benchmark*, *performance*, *throughput*, and *latency* over 2006–July 2026, followed by citation chaining, retaining empirical performance comparisons of relational access layers or engines and setting aside non-empirical or non-relational studies.

2.1. Object-relational mapping and the access-layer spectrum

Access layers exist to bridge the object-relational impedance mismatch between the row-and-table relational model and the object graphs an application manipulates [16, 17], spanning native drivers, query builders, and object-relational mappers that hydrate rows into objects and manage associations following patterns such as Data Mapper and Active Record [2] (Section 1). Torres et al. [18] survey two decades of ORM patterns and their implications for application design; the trade-off they document, developer productivity against a runtime cost that is hard to predict, is the one this paper quantifies. The same abstraction question recurs one level up in the polyglot-persistence debate over whether to adopt non-relational stores at all [19]; we hold that choice fixed at relational engines and vary only the access layer.

2.2. ORM performance and data-access anti-patterns

A parallel line of research studies ORM performance from the opposite direction: rather than benchmarking layers, it detects and repairs the inefficient SQL that mapping frameworks emit. The costliest pattern is the N+1 query problem, where iterating over a parent collection issues one child query per row; it and related redundant-query anti-patterns are pervasive across real-world web applications [20, 21], and Shao et al. [22] give an independent taxonomy and detector for database-access anti-patterns. Chen et al. quantify the performance impact of redundant data access in ORM-backed Java systems [23], extending earlier anti-pattern detection work [1]. A complementary strand shows the overhead is often *removable*: query synthesis lifts imperative accessor code into more efficient SQL [24], lazy evaluation batches round trips that would otherwise serialize [25], view-centric optimization rewrites whole request handlers [26], and

second-level caching absorbs repeated reads [27]; the analogous hazard in the JavaScript runtime is mis-sequenced `async`/`await` that serializes independent I/O [28]. Closest to our design, Lorenz et al. [29] show quantitatively that the best O/R mapping strategy depends on the underlying database technology, van Zyl et al. [30] that ORM performance is highly sensitive to tuning, and a recent study adds an energy dimension [31]. This literature establishes *where* ORM overhead concentrates (relation-materializing, N+1-prone access) and that it is configurable rather than fixed; our per-pattern results confirm and quantify both across the layers studied here, but it does not compare drivers, query builders, and ORMs head-to-head under controlled load, which is our aim.

2.3. Peer-reviewed access-layer comparisons

The peer-reviewed literature that measures access layers head-to-head is narrow. An early study by van Zyl et al. [32] weighed object databases against ORM tools, an ancestor of the present comparison. Colley et al. [8] give the earliest systematic cross-language account, benchmarking eight ORM frameworks across Java, C#, Python, and PHP against a common relational schema, but *deliberately excludes JavaScript*, leaving the Node.js ecosystem, and Express in particular, without an academic baseline for the better part of a decade. Three recent studies partially address this: one, in *Procedia Computer Science*, compares Prisma against optimized raw SQL over eight query patterns on a containerized PostgreSQL setup [9] (cited for its methodology; its headline speedup figures were not reproducible in our own setup, so we do not build on them); a second compares Prisma, TypeORM, and Sequelize across four relation types, also on PostgreSQL, by response time, memory, and CPU [10]; and the most recent instruments the same three ORMs on PostgreSQL inside a NestJS service, timing internal query stages rather than client-observed requests [11], independently observing the concurrency-dependent Prisma ranking flip we also report (worst at single queries, best under parallel load). All three are useful precedents for containerized, Node-specific measurement, but each covers at most three libraries, each is confined to PostgreSQL, and none measures the client-observed

HTTP round trip. A further Express-based study optimizes a single stack (pooling, batching, caching) rather than comparing access layers [14]. None of the peer-reviewed work located spans the native-driver, query-builder, and ORM tiers together, or includes MySQL.

2.4. Database-engine, SQL/NoSQL, and Node.js performance

A second, largely disjoint line of work benchmarks the layers immediately below and above the access layer without varying it directly. Salunke and Ouda [12] compare PostgreSQL and MySQL head-to-head under standard OLTP workloads (the only dual-engine precedent found), but benchmark the *engines themselves*, bypassing any application-level access layer, so their work cannot say how a driver, query builder, or ORM would change it. In the broader engine-comparison line, Li and Manoharan [33] contrast SQL and NoSQL databases, Gyorodi et al. [34] compare MongoDB against MySQL, and surveys of the scalable-datastore landscape frame when each class of store applies [35, 36]; all measure the store, not the access layer. On the Node.js side, Chaniotis et al. [3] established Node's viability as a web-application platform, Kuffel and Walter [4] tracked how Node.js back-end performance drifts under sustained load, do Carmo et al. [5] compared back-end frameworks more broadly, and Sun et al. [37] characterize how the single-threaded event loop schedules the asynchronous I/O every access layer relies on. None isolates the database access layer as an independent variable, let alone varies it across the taxonomy studied here; where database access appears, it is incidental to their designs rather than the object of study.

2.5. Standard benchmarks and benchmarking methodology

Standard benchmarks define how database systems are normally measured: YCSB [38] for cloud-serving stores and the TPC-C/TPC-E suites [39] for OLTP, with OLTP-Bench [40] providing a reusable, extensible harness. These target the *engine* and its transaction mix, saying nothing about the driver, query builder, or ORM through which an application issues work (the variable this study isolates),

but they set the methodological bar for controlled, repeatable load. A separate methodological literature explains why the vendor benchmarks' single-metric habit is a problem in its own right: Fruth et al. [7] and Raasveldt et al. [41] catalog pitfalls specific to database benchmarking (coordinated omission, warm-up, tool-induced distortion) that silently corrupt reported percentiles; Dean and Barroso [6] show why the tail, not the mean, determines user-perceived latency once a request fans out over many components, as an Express request serving a deep/nested fetch does; and Li et al. [42] trace the hardware, OS, and application sources of that tail. Load-testing and performance-testing surveys report such measurement is often ad hoc [43, 44], and repeatability studies find systems results frequently hard to reproduce [45], motivating a shared, reusable harness; the benchmarking literature more broadly stresses relevance, repeatability, and fairness [46] and the role of neutral benchmarks in advancing a field [47]. From this literature we take a design choice absent from every access-layer benchmark surveyed above: report throughput *and* tail latency (p99) for the same run, rather than a single metric family chosen after the fact.

2.6. Practitioner and vendor benchmarks

A larger body of evidence comes from practitioners and vendors, though each source is authored by a party with a stake in one of the compared products. Drizzle publishes both a Northwind micro-suite spanning nine targets (`pg`, Knex, Kysely, MikroORM, TypeORM, Prisma, and Drizzle itself, pooled and unpooled) and an official k6-driven load test reporting requests per second, average latency, P95, and CPU [13]. Prisma's own site compares Prisma, Drizzle, and TypeORM by median query latency over 500 iterations, reporting neither throughput nor any latency percentile [48]. IMDBench, maintained by Gel Data (formerly EdgeDB), adds Sequelize and node-postgres and reports both throughput and latency on three CRUD-style operations [49]. Together these three sources cover more of the taxonomy than the entire peer-reviewed literature combined, yet share the same defects: each is authored, or at least hosted, by a party selling one of the compared products; all run exclusively against Post-

greSQL; and each reports a single metric family, throughput or latency rarely both, almost never the tail.

2.7. Positioning

In the sources searched through July 2026 (search protocol archived with the replication package), no study combined broad taxonomy coverage, both engines, joint throughput/tail reporting, and vendor independence; each work found covers at most two or three of these four properties (Table 1). This study combines all four: it adds MySQL alongside PostgreSQL, spans a representative native-driver-to-ORM cross-section rather than the two or three libraries typical of peer-reviewed work, is authored by no evaluated vendor, reports throughput and tail latency (p99) jointly for every cell, and measures through the Express HTTP layer rather than in-process query timing alone, closing the same engine gap that separates access-layer studies from engine-only work such as [12].

Table 1: Prior work on relational-database access-layer performance, positioned along the four axes on which this study differs from it. "Layers covered" aggregates native drivers, query builders, and ORMs into one count; "none" marks studies that vary the engine or framework but not the access layer.

Study	Layers covered	Engines	Metrics	Independent?
Colley et al. [8]	8 ORMs, 4 languages (no JS)	not stated	query latency	Yes
Procedia CS [9]†	Prisma vs. raw SQL	PostgreSQL	latency, CPU	Yes
JCCT [10]	3 ORMs	PostgreSQL	latency, memory, CPU	Yes
JCSI [11]	3 ORMs	PostgreSQL	per-stage latency	Yes
Salunke and Ouda [12]	none (engine only)	PostgreSQL + MySQL	throughput, latency	Yes
Drizzle [13]	9 (broad)	PostgreSQL	throughput, latency, P95	No (vendor)
Prisma [48]	3 ORMs	PostgreSQL	median latency only	No (vendor)
IMDBench [49]	4 ORMs + driver	PostgreSQL	throughput, latency	No (vendor)
This study	**9 idiomatic (+2 tuned)**	**PostgreSQL + MySQL**	**throughput + tail (p99)**	**Yes**

†Cited for its methodology only; we could not reproduce the reported speedup figures in our own setup.

3. Study Design

Our goal, in Goal–Question–Metric terms, is to *analyze* the relational database access layer of an Express.js service *for the purpose of* quantifying its

performance cost *with respect to* throughput and tail latency *from the point of view of* an application developer choosing a layer *in the context of* PostgreSQL and MySQL back ends [50]. The design follows established guidelines for controlled experimentation in software engineering [51, 52], crossing seven portable access layers with both engines and all five patterns, plus two engine-specific native drivers and their tuned counterparts `pg-tuned` and `mysql2-tuned` (eleven layers: nine idiomatic, two tuned lower-bound baselines); the full rationale, including the pitfalls each control guards against, is distributed with the replication package (`METHODOLOGY.md`). Two estimands are kept separate throughout: RQ1–RQ3 (Section 4) answer the *default-configuration* estimand, the cost a practitioner incurs from each layer used idiomatically, while the same-SQL control below answers a *decomposition* estimand, what remains once every layer executes identical SQL.

3.1. Factors and treatments

We vary three factors. The *access layer* has eleven levels spanning the taxonomy: the native drivers `pg` and `mysql2`, their tuned counterparts `pg-tuned` (named prepared statements reused across calls) and `mysql2-tuned` (the binary protocol with server-side prepared statements via `execute()`), included as engine-specific lower-bound baselines rather than idiomatic treatments; the query builder Knex; and the ORMs Drizzle (a lightweight, query-builder-style ORM), Prisma, Sequelize, TypeORM, Objection, and MikroORM. We classify a library by its dominant interface, not its self-description: native drivers expose SQL and the wire protocol directly, query builders assemble SQL programmatically, and ORMs map rows to objects and manage associations; Drizzle sits at the query-builder end of the ORM range and Objection layers an ORM over Knex, which the results bear out. The nine idiomatic layers cover the taxonomy's three tiers but are a representative cross-section, not an exhaustive catalogue; other libraries (for example Kysely, Slonik, and pg-promise) are omitted. Vendor independence here means authorship only: no evaluated library's maintainers were involved or invited to inspect the adapters. It does not re-

move the consequential choices any benchmark author makes (which libraries to include, what counts as idiomatic use, the schema and endpoints, the pool size, allowed tuning); we disclose these throughout and release the harness for inspection. The *engine* has two levels, PostgreSQL 18.4 and MySQL 9.7.1. The *access pattern* has five levels, realized as HTTP endpoints (Table 2). The four native and tuned baselines are engine-specific (`pg`/`pg-tuned` run only on PostgreSQL, `mysql2`/`mysql2-tuned` only on MySQL); the other seven layers run on both engines, giving $4 + 7 \times 2 = 18$ layer $\times$ engine cells and $18 \times 5 = 90$ measured configurations.

Table 2: The five access patterns and what each stresses.

Endpoint	Pattern	Stresses
`GET /posts/:id`	point read (PK)	per-query overhead
`GET /posts?before=`	keyset range scan	index seek + hydration
`GET /posts/:id/thread`	deep/nested fetch	join strategy, N+1 avoidance
`GET /authors/:id/summary`	aggregation	grouped counts / raw SQL
`POST /posts`	insert	write path

3.2. Objects: schema, workload, and data

The workload is a minimal blog domain (`authors` $1{-}*$ `posts` $1{-}*$ `comments` $*{-}1$ `authors`), exercising a one-to-many relationship and a two-hop join for the deep fetch while remaining small enough to reason about. The five endpoints map onto canonical CRUD access patterns [38]: a primary-key point read; a newest-first *keyset* page (`WHERE id < cursor ORDER BY id DESC LIMIT` n, using the primary-key index rather than a large `OFFSET`); a deep/nested fetch returning a post with its author and every comment, each with its own comment-author; a per-author aggregation (post count, comment count, total views); and a single-row insert. The database is loaded from a deterministic seed (2,000 authors, 100,000 posts, 1,000,000 comments) by the native driver, independently of any layer under test, so every engine receives byte-identical data. The work-

ing set fits in memory, so measurements reflect access-layer cost rather than storage latency; once memory-resident, per-request instruction and logging cost dominate over disk I/O [53]. Every request draws its target identifier uniformly at random from the full seeded range (posts 1–100,000, authors 1–2,000), so load spreads across the whole working set rather than repeatedly hitting one hot record, and the read endpoints measure warm access-layer cost. A skewed, production-like key distribution is a planned variant (Section 6).

3.3. The adapter contract and N+1 control

Every access layer implements the same factory interface (five query methods plus a teardown), so the Express server and the load runner are layer-agnostic. Each adapter returns an *identical result shape*, and each uses that layer's *recommended idiomatic* mechanism to avoid the N+1 query problem on the deep fetch: a hand-written join for the native drivers, the Knex query builder, and the query-builder-style Drizzle (the tuned baselines reuse the native drivers' join unchanged), and relation eager loading (`include`, `withGraphFetched`, `populate`, or `relations`) for the data-mapper ORMs. The comparison is therefore one of *idiomatic usage*, not a single fixed query plan: each layer runs the code its documentation recommends, so a measured difference reflects the SQL it generates, its round-trip count, and how it materializes results. One asymmetry is inherent to any driver-versus-ORM comparison: the native side's idiom is hand-authored SQL, whereas the ORM side's idiom is the library's default; the same-SQL control below quantifies how much of the difference that choice accounts for. The number of round trips is itself part of what varies: the native drivers, Knex, and Drizzle issue a two-query deep fetch, whereas each ORM's eager-loading strategy selects its own query count and join shape. Server-side statement logging captures the exact SQL each layer emits, its round-trip count, and `EXPLAIN` plans for the native statements, released with the replication package; Supplement Table S2 tabulates the per-endpoint counts, which differ by design: the deep fetch takes one query in Sequelize and MikroORM, two in the native drivers, Knex, Drizzle, and TypeORM, and four in Prisma and Objection.

To separate a layer's execution machinery from its strategy choice, every adapter also implements a *same-SQL control*: identical two-statement deep-fetch SQL (and, per engine, identical `EXPLAIN` plans) with identical row mapping, executed through the layer's raw-SQL facility, mirroring the aggregation control (results in Section 4). Server-side statement capture, synchronized to a request marker so connection handshakes are excluded, confirms the two control statements are byte-identical across every layer on both engines; what differs is the wire protocol each driver negotiates for them: on PostgreSQL every layer uses the extended protocol with server-side prepared statements except MikroORM, which falls back to the simple protocol with client-side literal inlining, and on MySQL every layer uses the text protocol except `mysql2-tuned` and Prisma, which use the binary protocol with server-side prepared statements. To guarantee no adapter gains an unfair advantage by silently returning less, or by issuing N+1 queries, an automated cross-check (`bench/verify.mjs`) funnels every adapter's rows through shared canonical constructors and asserts full JSON byte equality against the native-driver baseline, on both engines, across 12 probes per adapter spanning the four non-mutating patterns and the same-SQL control, before any timing is collected. This check caught two defects during development (a fan-out aggregation bug and a driver result-shape mismatch; see Section 5).

3.4. Controls

Four controls isolate the access layer as the only variable. *(i) Pool size* is fixed at ten connections for every adapter, so the comparison measures access-layer overhead rather than pool tuning, a known web-application performance factor [54, 55]; for Prisma, whose default pool is otherwise derived from core count, this is pinned explicitly via the `connection_limit` URL parameter. Because the load generator opens 50 concurrent connections against this ten-connection pool, roughly forty in-flight requests are always waiting in each library's connection-acquisition queue; this queueing path is deliberately part of what we measure, since it is what a loaded production service exercises, and it is held structurally identical (same pool size, same demand) across layers. A 200 ms connection-pool

sampler corroborated this on the deep fetch: the native PostgreSQL cell ran with its ten-connection pool fully occupied (mean utilization $\approx$ 10 of 10) and a mean of about 32 requests pending in the acquisition queue (peaks of 39–40), evidence of queueing rather than an assumption about it. *(ii) Identical query semantics*: each endpoint issues the same logical query across layers; aggregation, in particular, uses a single correlated-subquery formulation everywhere (the documented raw-SQL path), so that endpoint measures layer overhead on an identical plan, not differences in generated SQL. *(iii) Write isolation*: because the insert endpoint mutates the dataset, every cell resets it before warm-up and before every measured run, not merely once per cell; the write endpoint is additionally measured in its own dedicated server boot, taken after the database's physical state is rebuilt rather than merely emptied by row deletion (PostgreSQL: drop and recreate the database from a seeded template; MySQL: delete the benchmark rows, rebuild the table with `OPTIMIZE TABLE`, and re-pin `AUTO_INCREMENT`), so sequences, physical layout, and purge state cannot drift across replicates or leak between adapters. *(iv) Observer separation*: the HTTP load generator runs in a process separate from the server.

3.5. Measurement procedure

Load was generated with `autocannon` [15], an HTTP/1.1 load tester issuing requests continuously at fixed concurrency and recording a latency histogram; it is a closed-loop (constant-concurrency) generator [56]. We measured each of the 90 configurations as 25 *independent replicates*. For each replicate we booted a fresh server process, opened 50 concurrent connections, ran a fifteen-second warm-up pass whose measurements were discarded, and then took one twelve-second measured run. The warm-up length follows a separate cold-boot measurement logging per-second throughput from server start for a fast, a middling, and the slowest PostgreSQL layer: every layer reached and sustained at least 95% of steady-state throughput within 12 s (Prisma by 5 s, the native driver by 9 s, MikroORM, the slowest, by 12 s), so the warm-up begins measurement past that ramp, absorbing JIT compilation, connection-pool fill, and

query-plan caching, since managed runtimes do not always reach a stable steady state promptly [57]. The few-percent residual fluctuation around the plateau is bounded separately by the sustained-load check in Section 6. The experimental unit is the freshly booted server run for one (layer, engine, endpoint) cell in one replicate; individual HTTP requests within a run are subsamples, not independent units. Booting a fresh process per replicate prevents persistent application-process state (JIT state, pool contents, heap) from carrying across replicates, though it does not guarantee full statistical independence: all runs share one host and one short campaign, so replicate index and measurement order are treated as blocking factors, not sources of independence. Cell order was reshuffled independently within each replicate; a forward-versus-reversed-order check on the write endpoint (5 replicates per direction) found no detectable history effect (reversed-to-forward throughput ratio across all 18 cells: 0.970–1.021, median 1.004), supporting the shuffled design. Within a replicate, every layer and engine is driven by the identical request-id sequence, generated by a PRNG seeded from the (endpoint, replicate) pair; this paired-stream design removes between-layer sampling noise as a confound, and the first ids of every stream are archived in the replication package. We report the median of the 25 replicate values; 449 of the 450 dedicated write-endpoint boots in the primary campaign completed normally, the exception being knex/PostgreSQL, whose replicate-16 boot hit a health-check timeout immediately after the physical database rebuild, leaving that cell with 24 of 25 replicates (the runner records and skips such failures rather than retrying silently). A port allocator that skips the ports the databases themselves listen on prevents the collision that had cost an earlier replicate its server process. Each replicate yields throughput (requests per second) and latency percentiles p50, p90, p97.5, and p99. During measured runs we sampled server CPU utilization and peak resident memory from `/proc` over the full process tree (parent plus descendants, so an in-process engine or spawned helper cannot escape measurement), with database and load-generator CPU sampled separately for comparison; a `perf_hooks` GC observer (event count, total pause time) and the 200 ms connection-pool

sampler ran inside the server, polled through a `/stats` endpoint after each run. Inserts were measured under each engine's *default* durability configuration (PostgreSQL `fsync=on`, `synchronous_commit=on`, `full_page_writes=on`; MySQL `innodb_flush_log_at_trx_commit=1`, `sync_binlog=1`, `log_bin=ON`), the regime a deployment runs unless an operator deliberately relaxes it; a labelled secondary run instead relaxed every one of those settings on both engines (10 replicates, write endpoint only) to isolate the durability mechanism, compared with the default regime in Section 5 (Supplement Table S3). A fixed-payload `/baseline` endpoint returning a fixed, seed-realistic thread-shaped payload measured the shared Express-plus-serialization floor (bounding framework and serialization cost, not the per-request object construction each layer performs): 10,909 (PostgreSQL server) and 10,921 (MySQL server) req/s, roughly 3× the fastest deep-fetch cell, so the framework floor leaves between-layer spreads visible rather than compressing them. The hardware and software environment was captured automatically into the replication package. To characterize behavior under load, we swept the deep/nested fetch (the most layer-sensitive pattern) across connection counts from 1 to 256 for every layer and engine, since throughput and contention scale non-linearly with concurrency [58]; a separate equal-CPU-budget control, confining the server process to 1, 2, or 4 cores, attributes part of that scaling directly to CPU (Section 4). A native, `undici`-based constant-arrival generator drove the deep fetch under a fixed offered rate (250–4,000 req/s) rather than fixed concurrency, measuring latency from each request's intended send time and clipping timed-out requests into the distribution at their cutoff; this coordinated-omission-corrected design [59, 60] independently validates the closed-loop tail results (Section 4).

3.6. Analysis

Because absolute throughput is hardware-specific, we analyze *relative* differences across layers within an engine. We quantify a pattern's sensitivity to the access layer by its *spread*: the idiomatic native driver's (`pg` or `mysql2`; the tuned baselines are a separate lower-bound reference, not the spread's anchor) median

throughput divided by that of the slowest layer on that pattern (native-relative, so comparable across patterns even where an ORM exceeds the native driver). We report medians over the 25 independent replicates, the coefficient of variation (CV) as a stability signal, following the practice of reporting central tendency and variability rather than a single number [61] and of using enough replicates to bound variance [62] (Section 4), and a bootstrap 95% CI for the median. The design is a randomized block: within each replicate every layer runs the identical request stream, so two layers' throughput samples are matched by replicate and must be compared with *paired* tests. To test whether adjacently ranked layers are distinguishable rather than an artifact of measurement noise, we compare each adjacent pair on the deep/nested fetch on their per-replicate throughput ratios with a two-sided paired sign-flip permutation test (20,000 permutations, so the smallest attainable p is $1/(20{,}001)$), cross-checked with the Wilcoxon signed-rank test. We report the geometric-mean paired ratio with a paired bootstrap 95% CI and the paired dominance (fraction of replicates in which the faster layer wins); this is confirmatory analysis over the seven adjacent-rank comparisons (the confirmatory family), to which we apply a Bonferroni correction. For the pg–Prisma pair we additionally report a paired two-one-sided-tests (TOST) equivalence result against an a-priori $\pm 5\%$ margin on the log-ratio (justified below). The layer$\times$engine interaction is tested per pattern with a blocked permutation test: for each layer and replicate we form the paired PostgreSQL-minus-MySQL log-throughput difference and permute layer labels within each replicate block (20,000 permutations), testing whether the engine effect varies across layers with replicate held as the block. Rank agreement across engines (Spearman, Kendall) is reported descriptively over the seven evaluated portable layers, the systems under study rather than a sample from a population, so we do not attach population confidence intervals to those correlations. Reporting nonparametric tests with standardized effect sizes follows recommended practice for randomized performance experiments in software engineering [63, 64]. We treat comparisons of widely separated layers as descriptive, not as additional confirmatory hypotheses. The $\pm 5\%$ equivalence margin was chosen a priori as

a practical-significance threshold (a throughput difference under 5% is below this study's own run-to-run variability, CV $\leq$ 7.6%), reported as a chosen, not a preregistered, margin; the equivalence conclusion is not sensitive to the exact value within a few percent.

3.7. Experimental setup

All measurements were taken on 2026-07-12 and 2026-07-13 on a single host: the primary matrix ran overnight across both dates, and the order-invariance, durability, same-SQL, open-loop, fan-out, equal-CPU, and concurrency-sweep runs followed on 2026-07-13 (a virtualized Intel Xeon Gold 6140 instance, 16 vCPUs, single NUMA node, 126 GB RAM; Linux 6.17; Node.js 24.16.0), with the database engines running locally. The pinned versions (Supplement Table S11) were selected and installed on the measurement date, recorded from the lockfile and an automated environment capture (replication package), and frozen for the study, since access-layer performance is version-sensitive; later point releases (for example Node.js 24.17.0) are deliberately excluded. We pin Express 4 rather than Express 5; since the framework version is shared by every cell, it shifts the common request-handling floor rather than the between-layer comparison to first order, though a different framework version could interact with a layer's scheduling or serialization, and re-running the harness on Express 5 is a one-line change.

3.8. Replication package

The harness (the Express application, the adapter contract, all nine idiomatic adapters and the two tuned baselines, the deterministic seed, the `autocannon` matrix runner, the correctness cross-check, and the scripts that regenerate every table in Section 4) is released so the full matrix can be re-run with a single command against a containerized or local PostgreSQL and MySQL.

4. Results

The tables below report throughput (requests per second, higher is better) and tail latency (p99, milliseconds, lower is better) per access layer and engine, one table per access pattern. All numbers come from the run described in Section 3: 25 independent replicates per cell, each a freshly booted server process, pool fixed at ten, inserts under each engine's default durability configuration. Each native driver is reported both idiomatically (`pg`, `mysql2`) and with reusable prepared statements (`pg-tuned`, `mysql2-tuned`); the tuned rows are a practitioner-oriented lower bound on what the wire protocol allows under this workload, not a ceiling every idiomatic layer could reach without code changes. Absolute throughput is hardware-specific, so the comparison of interest is *relative* across layers within an engine (Section 3). Two estimands recur below: the *default-configuration (idiomatic) effect* (the primary estimand behind RQ1–RQ3, realized in the five tables above), and the *same-SQL effect*, what remains when every layer executes identical SQL with identical row mapping (Table 8); the two are reported separately below.

4.1. RQ1: overhead relative to the native baseline

The native `pg` driver, or its tuned counterpart, leads only on the PostgreSQL point read and insert. On every other pattern-engine combination, Prisma leads outright, including over the tuned native baseline: the keyset range scan on both engines, aggregation on both engines, the point read on MySQL, and the deep fetch on MySQL (Tables 3, 4, 5, 6). It does so at roughly five times the application-side CPU of every other layer (498% on the deep fetch; Supplement Table S10), so the finding is a CPU-for-throughput trade, not a free win. The remaining PostgreSQL deep fetch is close enough between `pg` and Prisma to need a formal equivalence test (below). The cost of abstraction remains strongly *pattern*- and, now visibly, *engine*-dependent.

The tuned baselines sharpen what "native" means as a ceiling. On PostgreSQL, `pg-tuned` tops both the point read (6,851 vs. `pg`'s 6,695 req/s) and

Table 3: Point read: throughput (req/s, higher is better) and tail latency (p99, ms, lower is better) by access layer and engine.

		PostgreSQL		MySQL	
Layer	Category	req/s	p99	req/s	p99
pg	native-driver	6695	9.0	–	–
mysql2	native-driver	–	–	4337	16.0
pg-tuned	native-tuned	6851	9.0	–	–
mysql2-tuned	native-tuned	–	–	4302	14.0
knex	query-builder	5096	12.0	3837	15.0
drizzle	orm-lightweight	4555	13.0	3168	20.0
prisma	orm	6051	12.0	5993	11.0
sequelize	orm	3532	16.0	2746	21.0
typeorm	orm	4331	14.0	3517	16.0
objection	orm	4070	15.0	3222	18.0
mikroorm	orm	2185	26.0	1839	31.0

Table 4: Keyset range scan: throughput (req/s, higher is better) and tail latency (p99, ms, lower is better) by access layer and engine.

		PostgreSQL		MySQL	
Layer	Category	req/s	p99	req/s	p99
pg	native-driver	3782	18.0	–	–
mysql2	native-driver	–	–	3231	21.0
pg-tuned	native-tuned	3811	17.0	–	–
mysql2-tuned	native-tuned	–	–	3216	18.0
knex	query-builder	3163	19.0	2952	20.0
drizzle	orm-lightweight	2905	20.0	2265	28.0
prisma	orm	3963	17.0	3936	18.0
sequelize	orm	2426	24.0	2074	27.0
typeorm	orm	2546	25.0	2474	24.0
objection	orm	2669	22.0	2514	23.0
mikroorm	orm	773	74.0	729	79.0

Table 5: Deep/nested fetch: throughput (req/s, higher is better) and tail latency (p99, ms, lower is better) by access layer and engine.

		PostgreSQL		MySQL	
Layer	Category	req/s	p99	req/s	p99
pg	native-driver	3741	20.0	–	–
mysql2	native-driver	–	–	2405	29.0
pg-tuned	native-tuned	3813	18.0	–	–
mysql2-tuned	native-tuned	–	–	2436	25.0
knex	query-builder	2491	27.0	2050	30.0
drizzle	orm-lightweight	2080	31.0	1446	43.0
prisma	orm	3685	18.0	3701	18.0
sequelize	orm	989	60.0	893	65.0
typeorm	orm	1327	45.0	1170	50.0
objection	orm	1037	55.0	860	67.0
mikroorm	orm	571	106	535	113

Table 6: Aggregation: throughput (req/s, higher is better) and tail latency (p99, ms, lower is better) by access layer and engine.

		PostgreSQL		MySQL	
Layer	Category	req/s	p99	req/s	p99
pg	native-driver	7035	10.0	–	–
mysql2	native-driver	–	–	4195	17.0
pg-tuned	native-tuned	7731	9.0	–	–
mysql2-tuned	native-tuned	–	–	4194	17.0
knex	query-builder	5754	12.0	3945	17.0
drizzle	orm-lightweight	6427	11.0	3944	18.0
prisma	orm	7893	9.0	7383	9.0
sequelize	orm	4807	15.0	3390	19.0
typeorm	orm	6127	12.0	4371	17.0
objection	orm	3775	18.0	2963	22.0
mikroorm	orm	5279	14.0	3590	19.0

Table 7: Insert: throughput (req/s, higher is better) and tail latency (p99, ms, lower is better) by access layer and engine.

		PostgreSQL		MySQL	
Layer	Category	req/s	p99	req/s	p99
pg	native-driver	6064	11.0	–	–
mysql2	native-driver	–	–	809	138
pg-tuned	native-tuned	6280	12.0	–	–
mysql2-tuned	native-tuned	–	–	808	135
knex	query-builder	4693	14.0	795	135
drizzle	orm-lightweight	4230	14.0	796	138
prisma	orm	5465	14.0	714	154
sequelize	orm	2668	21.0	772	135
typeorm	orm	2595	24.0	732	152
objection	orm	3926	15.0	787	140
mikroorm	orm	1571	37.0	663	153

the insert (6,280 vs. 6,064), and raises the deep-fetch ceiling slightly further (3,813 vs. 3,741); reusable prepared statements are worth a few percent on PostgreSQL. On MySQL, `mysql2-tuned` is within noise of idiomatic `mysql2` on the point read, range scan, aggregation, and insert (differences of at most 0.8%), except on the deep fetch, where it reaches 2,436 req/s against `mysql2`'s 2,405: the binary protocol and server-side prepared statements (wire-protocol evidence below) pay off only once a request issues more than one round trip.

On the **point read**, PostgreSQL ranges from `pg-tuned`'s 6,851 down to MikroORM's 2,185 req/s; the native-relative spread (`pg`'s idiomatic 6,695 divided by the slowest layer) is 3.06× (95% CI [2.97–3.16]). MySQL's spread is smaller, 2.36× (`mysql2` 4,337 down to MikroORM 1,839; CI [2.31–2.42]), but MySQL's fastest layer is not the native driver: Prisma leads at 5,993 req/s, 38% above `mysql2`.

On the **deep/nested fetch** (the pattern that assembles a post, its author, and all comments with their authors), the native-relative spread is the widest measured: 6.55× on PostgreSQL (`pg` 3,741 down to MikroORM's 571 req/s; CI [6.46–6.65]) and 4.50× on MySQL (`mysql2` 2,405 down to MikroORM's 535; CI [4.41–4.59]). Prisma sits close behind `pg` on PostgreSQL (3,685) and ahead of `mysql2` on MySQL (3,701, 54% above native). Because every layer runs the identical per-replicate request stream, the comparison is paired (Section 3); `pg` is faster in 16 of 25 replicates, by a geometric-mean ratio of 1.02 (paired 95% CI [1.00–1.04]; Table 9), a difference the paired sign-flip permutation test does not resolve as significant ($p = 0.058$; the Wilcoxon signed-rank test agrees, $p = 0.08$). A paired two-one-sided-tests (TOST) equivalence test at an a-priori ±5% margin confirms the two are equivalent (ratio 1.02, 90% CI [1.004–1.037], inside the ±5% band), and Prisma's p99 is the lower of the two (18 ms vs. 20 ms). We therefore read the top of the PostgreSQL deep-fetch ranking as a practical tie. Knex (2,491 / 2,050) and the lightweight ORM Drizzle (2,080 / 1,446) remain competitive on both engines, whereas the data-mapper ORMs trail by up to 6.55× on PostgreSQL: TypeORM, Sequelize, Objection, and especially MikroORM.

Aggregation overhead shrinks once every layer is forced onto the same raw correlated-subquery plan, but Prisma leads it outright on both engines, including over the tuned native baseline: 7,893 req/s on PostgreSQL (ahead of `pg-tuned`'s 7,731 and `pg`'s 7,035) and 7,383 on MySQL, 76% above `mysql2`'s 4,195. Excluding Prisma, the native-relative spread (`pg` or `mysql2` divided by the slowest layer, Objection on both engines) is 1.86× on PostgreSQL (CI [1.83–1.89]) and 1.42× on MySQL (CI [1.40–1.44]). Abstraction is thus inexpensive when a layer merely forwards a query, and on this pattern and the keyset range scan, Prisma's query engine turns that low cost into an outright lead; abstraction is costly only when the layer must materialize a nested object graph on the application side.

4.2. Same-SQL control

The idiomatic deep fetch lets each layer choose its own query strategy, so its spread mixes raw SQL execution with everything a layer does around it. To bound the raw-execution component, we re-measured the deep fetch under a *same-SQL control*: every layer executes identical two-statement SQL (and, per engine, identical `EXPLAIN` plans) with identical row mapping through its raw-SQL facility, as the aggregation endpoint already does (Table 8). Server-side capture confirms the statement text is byte-identical across layers on both engines, while the wire protocols differ per layer as enumerated in Section 3; the honest description of this control is therefore *same SQL, protocol disclosed*, not *same plan*. The contrast is composite: switching from a layer's idiomatic relational-fetch path to the common raw-SQL-and-shared-mapping path changes its query strategy, query count, prepared-statement behavior, the raw versus ORM API, and result marshalling together, so it isolates none of these individually; what it establishes is that the raw-execution component is small and the idiomatic overhead lies upstream of it.

On the identical SQL, the deep-fetch spread collapses to a max/min ratio (across the nine layers; the idiomatic spreads above are native-relative, whose max/min counterparts are 6.7× and 6.9×) of 1.76× on PostgreSQL (top:

Prisma's raw path at 4,342 req/s; bottom: Sequelize's raw facility at 2,472) and 2.21× on MySQL (Prisma 4,220; Sequelize 1,912). Prisma's raw path leads on both engines, consistent with its pipelined query engine; the new low end is Sequelize's raw facility (`sequelize.query`), which we attribute to dispatch and marshalling overhead in the raw API itself, not SQL execution. MikroORM shows the largest strategy gap: its idiomatic path (571 req/s on PostgreSQL, 535 on MySQL) reaches only 0.20 and 0.25 of its own same-SQL throughput (2,869 and 2,172), a 5.0× and 4.1× gap between its idiomatic and raw-SQL paths. On this pattern, the access layer's idiomatic cost is dominated by what it *chooses to do* by default (its query strategy, round-trip count, and object hydration), not by the speed of its raw SQL execution; the fixed-payload Express-plus-JSON floor (Section 3) sits well above every same-SQL cell, so the framework floor does not manufacture this collapse.

4.3. RQ2: engine dependence of the ranking

The relative ordering of access layers is largely, but not uniformly, stable across engines. The Spearman rank correlation between the PostgreSQL and MySQL orderings of the seven portable layers (reported descriptively, since these seven are the evaluated systems rather than a sample from a population; Section 3) is $\rho = 0.89$ on the point read, 0.89 on the range scan, 0.96 on the deep fetch, and 0.89 on aggregation (Kendall's τ between 0.81 and 0.90 on all four), so a layer's rank on one engine tracks its rank on the other for every read pattern. The **insert** ranking is the exception ($\rho = 0.43$, $\tau = 0.43$). Prisma illustrates why: it is the fastest ORM on PostgreSQL inserts (5,465 req/s, ahead of Objection's 3,926) but the second-slowest layer on MySQL inserts (714, ahead of only MikroORM's 663), a rank flip a benchmark run on either engine alone would not reveal. Beyond order, the *magnitude* of a layer's cost also depends on the engine: a blocked permutation test on the paired PostgreSQL-minus-MySQL log-throughput difference (Section 3) finds the layer×engine interaction significant on all five patterns ($p \leq 5 \times 10^{-5}$, the permutation-resolution floor, in each case), so single-engine results are not portable in degree even where they are

Table 8: Decomposing the deep-fetch spread: throughput (req/s) of the idiomatic deep fetch versus the *same-SQL* control (median of 10 replicates), in which every layer executes the identical two-statement SQL with identical row mapping through its raw-SQL facility. The ratio (idiomatic ÷ same-SQL) measures the gap between each layer's idiomatic relational-fetch path and the common raw-SQL path, a composite of query strategy, query count, protocol, the raw versus ORM API, and result marshalling and hydration changed together; the residual same-SQL spread (native-relative 1.47× on PostgreSQL, 1.27× on MySQL) bounds the raw-execution component.

	PostgreSQL			MySQL		
Layer	idiom.	same-SQL	ratio	idiom.	same-SQL	ratio
`pg`	3741	3643	1.03	–	–	–
`mysql2`	–	–	–	2405	2419	0.99
`knex`	2491	2906	0.86	2050	2305	0.89
`drizzle`	2080	3446	0.60	1446	2105	0.69
`prisma`	3685	4342	0.85	3701	4220	0.88
`sequelize`	989	2472	0.40	893	1912	0.47
`typeorm`	1327	2974	0.45	1170	2517	0.46
`objection`	1037	2845	0.36	860	2243	0.38
`mikroorm`	571	2869	0.20	535	2172	0.25

portable in order.

Prisma's engine-dependent lead is visible on the **keyset range scan** (Table 4): it leads on both engines (3,963 on PostgreSQL, 3,936 on MySQL), ahead of `pg-tuned` (3,811), `pg` (3,782), and `mysql2` (3,231). The native-relative spread is 4.89× on PostgreSQL (CI [4.80–4.99]) and 4.43× on MySQL (CI [4.35–4.54]), confirming that the earlier collapse observed under `OFFSET` pagination was a workload artifact rather than an engine or layer property (Section 5). Most of that spread traces to a single layer: MikroORM's range-scan throughput is 773 req/s on PostgreSQL and 729 on MySQL (p99 of 74 and 79 ms), well below every other layer on either engine, an anomaly we flag but do not fully isolate on this endpoint; it is consistent with the per-request cost of materializing the returned page into MikroORM's identity map, the same hydration overhead the same-SQL control isolates on the deep fetch (ratio 0.20 on PostgreSQL, above).

Inserts are the pattern where the engine dominates most visibly, and where the durability regime matters to that claim. All headline write numbers here are measured under each engine's *default* durability configuration (Section 3). On MySQL, insert throughput is low and tightly clustered across all nine layers (663–809 req/s, a 1.22× spread, CI [1.20–1.24]; Table 7) and its p99 sits at roughly 135–155 ms for every layer (a per-commit redo-log and binlog flush), whereas PostgreSQL spreads 3.86× (`pg` 6,064 down to MikroORM 1,571, CI [3.77–3.98]). A default-versus-relaxed comparison ($n = 25$ vs. $n = 10$ medians; Supplement Table S3) shows this is not a durability artifact: at 50 concurrent connections, group commit amortizes most of the per-commit flush cost, so relaxing durability buys at most 14% on PostgreSQL (native; under 8% on the layer-bound ORMs, median +8% across all nine rows) and 19–24% on MySQL (median +20%); if anything, the engine contrast is slightly larger under default durability than under the relaxed configuration used elsewhere. On MySQL, insert throughput is therefore governed mainly by the engine rather than the access layer; on PostgreSQL the access layer remains the larger factor. Which layer is fastest, however, does not transfer between the two: the ranking, not just the magnitude, is engine-dependent for writes in a way it is not for any

read pattern.

4.4. RQ3: which patterns matter most

Ranking the five patterns by native-relative spread on PostgreSQL gives deep fetch (6.55×) > range scan (4.89×) > insert (3.86×) > point read (3.06×) > aggregation (1.86×). The same deep > range > point > aggregation order holds on MySQL (4.50×, 4.43×, 2.36×, 1.42×); only the insert moves, from third place on PostgreSQL to last on MySQL (1.22×): the pattern where MySQL's engine floor, not the access layer, sets the ceiling (RQ2 above). The deep/nested fetch is the most consequential pattern on both engines; aggregation is the least, once every layer runs the same raw-SQL plan (the range scan's large nominal spread, as above, is driven disproportionately by MikroORM alone).

The primary deep-fetch workload uses posts with roughly ten comments on average, near the low end of a fan-out sweep across 0, 1, 10, 50, 100, and 500 comments (PostgreSQL, medians of three replicates) that shows the layer spread widening with graph size: 5.9×, 5.8×, 7.1×, 9.8×, 11.4×, and 16.1× respectively; the leader flips from Prisma at small graphs (up to 10 comments) to `pg` at 50 comments and above, as hydration cost scales with the number of materialized rows, and MySQL shows the same qualitative shape. The deep-fetch spreads reported above are therefore conservative: a larger graph would widen the gap between the lightest and heaviest layers further.

4.5. Tail latency

We report p99 for every pattern (the tail metric that governs perceived latency under load); p50, p90, and p97.5 are in the replication package. Tail latency tracks throughput: on the deep/nested fetch, PostgreSQL's p99 ladder runs `pg` 20, Prisma 18, Knex 27, Drizzle 31, TypeORM 45, Objection 55, Sequelize 60, and MikroORM 106 ms (Table 5), the same order as the throughput ranking, so a heavyweight ORM's nested-fetch penalty is visible not only in mean capacity but in the tail under load, a quantity single-metric, throughput-only vendor benchmarks omit.

These closed-loop p99 values come from a constant-concurrency generator. To check that the tail ranking is not an artifact of that model, we drove the deep/nested fetch on PostgreSQL under the coordinated-omission-corrected open-loop harness described in Section 3 (timed-out requests, a 10 s hard cap, clipped into the distribution; full sweep in Supplement Table S6). Below saturation the corrected tails are flat and small: `pg` and Prisma hold p99 at 2–4 ms up to 2,000 req/s, both sustaining the full offered rate, and Knex's p99 is still only 33 ms at 2,000. Beyond each layer's capacity the tail collapses rather than degrading gracefully: at the 4,000 req/s tier, `pg` achieves 3,697 of the offered requests with a corrected p99 of 1.20 s, Prisma achieves 3,501 (2.11 s), and Knex achieves 2,448 (9.4 s); Sequelize already collapses at 2,000 req/s (achieving 772, with a third of the issued requests timing out: 9,992 of 30,000), and MikroORM collapses at 1,000 req/s (achieving 488) and again at 4,000 (achieving 259, with 53,315 of 60,000 requests timing out). We call a layer's behavior at an offered rate a *collapse* when it achieves under half the offered rate with a large share of requests timing out. By that criterion the ranking matches the closed-loop result and sharpens it: the data-mapper ORMs enter unstable queueing with high timeout rates as soon as the offered load crosses their capacity, whereas the lightweight layers degrade gradually over a wider range.

4.6. Measurement stability and significance

The rankings above rest on stable measurements. Across the 40 PostgreSQL cells the coefficient of variation of throughput over the 25 independent replicates is at most 7.6% (`pg`, insert; Supplement Table S9), and across the 40 MySQL cells at most 6.6% (Knex, insert; Supplement Table S1); the largest relative median absolute deviation is 5.4% (`pg`, insert, PostgreSQL). These maxima are well below the between-layer spreads reported above (up to 6.55×). One cell, Knex/PostgreSQL/insert, has $n = 24$ rather than 25, the one write boot in the primary campaign that failed its post-rebuild health check (Section 3). A separate order-randomization check (forward versus reversed cell order, $n = 5$ each, insert endpoint) found no detectable history effect (Section 3).

To confirm that neighboring ranks are not measurement artifacts, we compare each adjacently ranked pair on the deep/nested fetch *pairwise* with the paired permutation test described in Section 3, reporting the geometric-mean paired ratio, a paired bootstrap 95% CI, and the paired dominance (Table 9); the Wilcoxon signed-rank test agrees throughout (replication package). Six of the seven adjacent pairs separate at the permutation-resolution floor ($p \leq 5 \times 10^{-5}$) with the faster layer ahead in every one of the 25 replicates (dominance 1.00, except objection > sequelize at 0.88), all with ratios at least 1.05. This is confirmatory analysis over the seven adjacent-rank comparisons (the confirmatory family), and every one of the six survives a Bonferroni correction across the family. The seventh, the top pair `pg` and Prisma, is the exception treated above: pg leads in 16 of 25 replicates by a geometric-mean ratio of 1.02 (95% CI [1.00–1.04]), which the paired permutation test does not resolve as significant ($p = 0.058$) and the paired TOST finds equivalent within ±5%. We therefore read the top of the ranking as a practical tie, and every step below it as both statistically and practically supported.

4.7. Scaling with concurrency

Figure 1 sweeps the deep/nested fetch from 1 to 256 connections (a separate sweep on PostgreSQL). The coarse three-band ordering, native and tuned-native together with Prisma on top, the query builder and the lightweight ORM in a middle band, and the data-mapper ORMs below, holds at every operating point at or above 32 connections; at 8 connections Prisma's engine is still mid-pack (1,869 req/s, below Knex's 2,438), joining the top band only from 32 upward. Within the top band, `pg` and `pg-tuned` lead Prisma at every concurrency from 8 upward (for example, at 256 connections: `pg-tuned` 3,709, `pg` 3,511, Prisma 3,203), so Prisma's near-native parity at the primary 50-connection operating point does not extend to an outright lead at higher concurrency in this sweep. At a single connection the ranking differs from the higher-concurrency regime (the workload is latency- rather than throughput-bound at that point), and Prisma is among the slowest layers there (503 req/s). The lightweight layers

reach a substantially higher throughput ceiling than the data-mapper ORMs across the whole load range, so the RQ1 finding is not an artifact of the single 50-connection operating point.

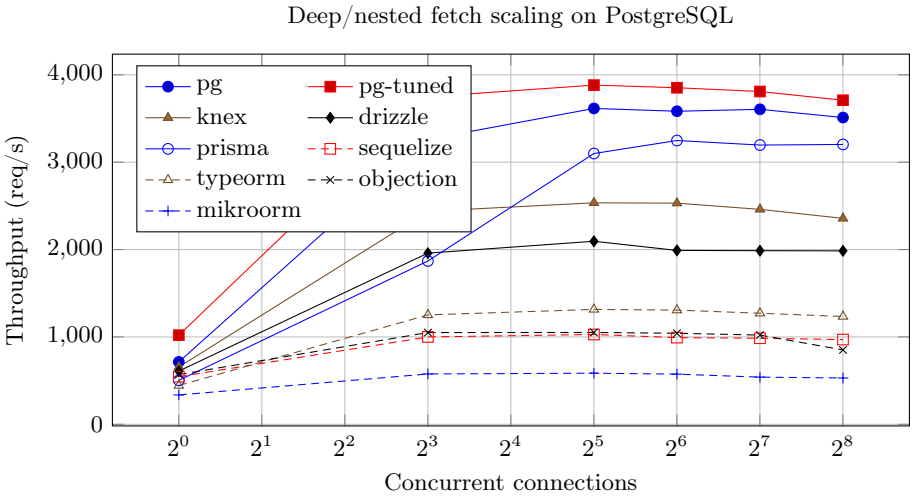

Figure 1: Throughput of each access layer on the deep/nested fetch as concurrency increases (postgres). The three-band ordering (native and tuned-native with Prisma on top, query builder and lightweight ORM in the middle, data-mapper ORMs below) holds at 32 connections and above; at 1 and 8 connections Prisma is mid-pack (Section 4).

4.8. Resource footprint

Throughput is not the only cost (Supplement Table S10). Prisma's near-native or leading throughput on several patterns comes at a much higher CPU cost: measured at the *same* ten-connection pool as every other layer, its Node.js process draws 498% (about five cores) on the deep fetch against the ≈ 100% (one core) of the other layers, because Prisma runs a multi-threaded query engine in-process (Section 3), a property of the layer rather than of a larger connection pool; the multiplier is pattern-dependent (Prisma's aggregation cell draws only ≈ 200%).

Normalizing throughput by CPU cost makes the trade explicit, but the normalization must be read at the right tier. Counting only *application*-process

CPU on the deep fetch, requests per CPU-second are `pg` 3,741 down to Prisma 740 and MikroORM 571, so Prisma ranks second-to-last. That figure understates the compute of layers that push work to the database, however: `pg`'s hand-written join drives about 3.6 database cores while Prisma's engine keeps more work in the application tier (Supplement Table S5). Counting *combined* application-plus-database CPU per completed request, `pg`'s efficiency advantage over Prisma shrinks from about 5× (3,741 vs. 740 req per application-CPU-second) to about 1.4× (813 vs. 596 req per combined-CPU-second); Prisma remains among the least efficient layers end-to-end, but the gap is a small multiple, not a large one. The application-tier reading is the one that matters when the application container's CPU is the budgeted resource; the combined reading is the end-to-end compute view. An equal-CPU control corroborates the application-tier trade operationally: confining the server process with `taskset` to 1, 2, or 4 cores (database and load generator pinned to disjoint cores; Supplement Table S4), `pg` reaches its full deep-fetch throughput on a single core (3,663, 3,277, and 3,700 req/s at 1, 2, and 4 cores), whereas Prisma needs roughly four cores to approach that level (896, 1,726, 3,025), 4.1× slower than `pg` on an equal one-core application budget; MikroORM stays low regardless of core count (463, 577, 542).

Memory differences are smaller. The native driver, Knex, and Objection are lightest (204–211 MB peak RSS), while the remaining layers use more (248–271 MB). Resource cost, like throughput, is pattern- and layer-specific, and matters when sizing horizontally scaled deployments, where per-instance resource use governs how many replicas a given load requires [65].

5. Discussion

Implications for practice.. Read together, RQ1 and RQ3 point to the same conclusion: abstraction cost is not a fixed, uniform overhead but is specific to the *access pattern*, and it concentrates in the pattern that assembles a nested object graph. Two estimands separate throughout this discussion: the *default-*

Table 9: Paired comparison of adjacently ranked access layers on the deep/nested fetch (PostgreSQL), over the 25 independent replicates. Because every layer runs the identical per-replicate request stream, layers are compared on their per-replicate throughput ratios: median throughput (req/s) with bootstrap 95% CI, the geometric-mean paired ratio A/B with a paired bootstrap 95% CI, paired dominance (fraction of replicates in which A exceeds B), and the two-sided paired sign-flip permutation p (20000 permutations; the Wilcoxon signed-rank test agrees, replication package).

Comparison (A > B)	med. A [95% CI]	med. B [95% CI]	ratio A/B [95% CI]	dom.	perm. p
pg > prisma	3741 [3687–3782]	3685 [3675–3712]	1.02 [1.00–1.04]	0.64	0.0584
prisma > knex	3685 [3653–3712]	2491 [2442–2534]	1.47 [1.45–1.50]	1.00	≤5e-5
knex > drizzle	2491 [2442–2534]	2080 [2048–2103]	1.20 [1.18–1.22]	1.00	≤5e-5
drizzle > typeorm	2080 [2048–2103]	1327 [1310–1343]	1.55 [1.53–1.57]	1.00	≤5e-5
typeorm > objection	1327 [1315–1343]	1037 [1028–1059]	1.28 [1.27–1.30]	1.00	≤5e-5
objection > sequelize	1037 [1026–1059]	989 [978–1007]	1.05 [1.03–1.06]	0.88	≤5e-5
sequelize > mikroorm	989 [978–1007]	571 [562–576]	1.74 [1.71–1.77]	1.00	≤5e-5

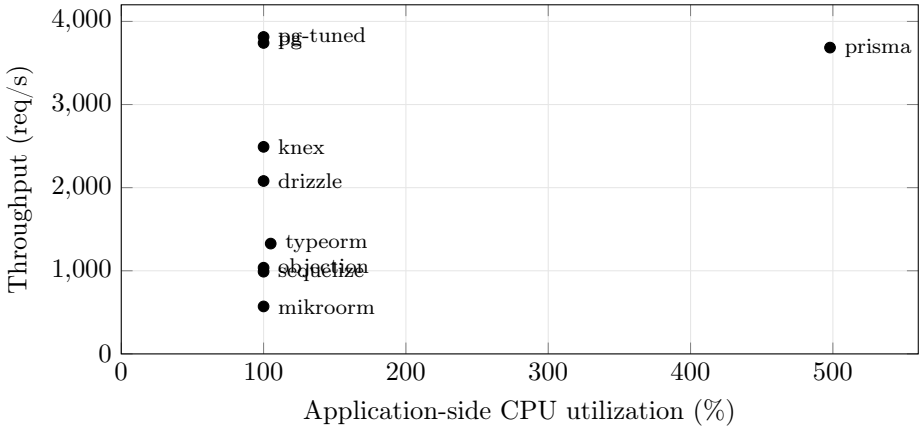

Figure 2: Throughput versus application-side CPU on the deep/nested fetch (PostgreSQL, ten-connection pool, medians of 25 replicates). Prisma reaches native-tied throughput at roughly five times the CPU of every other layer; the data-mapper ORMs are low on both axes. CPU is the server process only; database-side CPU is excluded for all layers alike.

configuration (idiomatic) effect a practitioner gets from each layer's documented defaults, and the *controlled same-SQL* effect that remains once every layer executes identical SQL. Under the default-configuration estimand, the deep/nested fetch requires the access layer to hydrate rows into objects and resolve associations (the object-relational impedance mismatch [16] made concrete), not merely to issue SQL, and this is where the data-mapper ORMs, Sequelize, TypeORM, Objection, and especially MikroORM, are most expensive, trailing the native `pg` driver by up to 6.55× on PostgreSQL and 4.50× on MySQL, the widest spreads in the study, with the tail showing the same pattern (Table 5, Section 4). A team whose hot path assembles nested object graphs therefore pays the largest abstraction cost on its highest-value traffic, a cost that grows, not shrinks, with graph size: the fan-out sweep (Section 4) shows the spread widening monotonically from 5.9× at zero comments to 16.1× at 500, with the leader flipping from Prisma on small graphs to `pg` from 50 comments on; the primary workload's roughly ten-comment graph sits near the low end of that range. Point reads and aggregation are the opposite case, dominated by connection and parsing cost every layer shares, spreading only 3.06×/2.36× and 1.86×/1.42× respectively once every adapter, native drivers included, runs the same raw correlated-subquery plan, though Prisma's raw-SQL path still beats the slowest layer, Objection, by 2.09× (PostgreSQL) to 2.49× (MySQL). On the controlled same-SQL estimand, the idiomatic deep-fetch spread collapses to 1.76× on PostgreSQL and 2.21× on MySQL (Table 8), so the wide default-configuration deficits lie upstream of raw SQL execution: in each layer's query-generation and result-construction path (its default eager-loading strategy, query count, and object hydration), which the control changes as a bundle rather than isolating one component. MikroORM illustrates this most starkly, its default path delivering only 0.20 of its own same-SQL throughput on PostgreSQL and 0.25 on MySQL, while Prisma's raw path is the fastest same-SQL cell on both engines; the same control exposes a new low end, Sequelize's raw-SQL facility (`sequelize.query`), evidence of dispatch and marshalling overhead in its raw API rather than in query strategy. Round-trip count alone does not set throughput: Prisma and Objection both issue four

deep-fetch queries (Supplement Table S2), yet sit at opposite ends of the default-configuration ranking, while Knex and Drizzle sit in a consistent middle ground on every pattern (never fastest, never among the slowest data-mapper ORMs), which is what a query builder and a non-data-mapper, lightweight ORM should deliver.

Same SQL, different wire protocols.. Server-side capture confirms the same-SQL collapse is not a protocol artifact: every layer emits byte-identical control statements on both engines, though the wire protocols differ per layer (Section 3). Protocol choice alone does not explain the same-SQL spread: Prisma leads it on MySQL using the binary protocol, and `mysql2-tuned`'s binary path sits within noise of plain `mysql2`'s text path on most patterns.

The CPU cost behind parity.. Prisma shows that an ORM need not be slow, consistent with evidence that ORM cost is highly sensitive to implementation and tuning [30]. On the deep fetch its compiled query engine is statistically equivalent to the native driver (paired ratio 1.02, permutation $p = 0.058$, TOST equivalence within $\pm 5\%$; Section 4), and it leads outright on the keyset range scan, the raw-SQL aggregation, and the point read and deep fetch on MySQL (Section 4). That parity or leadership is not free: at the *same* ten-connection pool, Prisma's process draws roughly five times the application-side CPU of every other layer on the deep fetch (498%; Supplement Table S10, Figure 2), trading CPU for latency [31], a cost a throughput-only benchmark would miss. How large that cost is depends on which CPU one counts: by *application*-tier CPU the efficiency ranking inverts sharply, but the native driver pushes about 3.6 database cores of join work that figure ignores; counting *combined* application and database CPU per request narrows `pg`'s advantage to about 1.4× (813 vs. 596 req per combined-CPU-second; Supplement Table S5). The honest reading is therefore tier-specific: Prisma spends several times the application CPU of the other layers, which matters directly when the application container's CPU is the budgeted resource (a shared host, a per-core billing model), yet end-to-end it is only modestly less compute-efficient than the native driver. The equal-

CPU control (Section 4) confirms the application-tier trade operationally: at an equal one-core application budget Prisma is 4.1× slower than `pg` and needs about four cores to approach parity, while MikroORM barely moves with more cores, confirming its deficit is a hydration cost rather than a parallelism-starved one.

Queueing under load and the open-loop check.. The pool sampler turns a previously inferential design property into a direct measurement: the native PostgreSQL driver's ten-connection pool sits fully utilized on the deep fetch under the 50 concurrent connections used throughout, confirming sustained queueing rather than an idle pool (Section 3). A coordinated-omission-corrected, constant-arrival cross-run of the deep fetch on PostgreSQL (Supplement Table S6) confirms the closed-loop ranking transfers under that stricter model: below capacity the corrected tail is flat and small, but once the offered rate crosses a layer's capacity the heavy data-mapper ORMs collapse rather than degrade gracefully (Section 4), so a closed-loop benchmark alone underestimates how abruptly, not just how much, a heavyweight ORM's tail degrades beyond capacity.

Writes: an engine-bound cost under default durability.. RQ2 has a clear answer on inserts, and the headline figures come from each engine's *default* durability configuration (Section 3) rather than the relaxed one used in earlier campaigns. Even with fsync on, PostgreSQL's insert throughput remains layer-dependent: `pg-tuned` and `pg` lead, Prisma and Knex stay competitive (5,465 and 4,693 req/s), and MikroORM trails, a 3.86× spread (Table 7). Group commit absorbs most of the per-commit flush cost at the 50-connection concurrency used throughout: a secondary, symmetrically relaxed run ($n = 10$, both engines, including MySQL's binlog sync off) shows relaxing durability buys native PostgreSQL at most 14% and at most a few percent on most layer-bound ORMs (Supplement Table S3), so the layer-dependence is a property of the write path, not of fsync. MySQL inserts, by contrast, remain low and flat across all nine layers, a 1.22× spread, the same "engine floor" pattern the relaxed-durability

run already showed; what default durability newly exposes is the mechanism in the tail: MySQL's p99 sits at 135–155 ms across every layer, consistent with the per-commit double flush (redo log, then binlog) InnoDB performs on every insert regardless of how the request reaches it [53], against 11–37 ms on PostgreSQL. Relaxing MySQL's durability recovers only 19–24%, and the absolute gap to PostgreSQL remains roughly 7× even then (966 vs. 6,895 req/s under the relaxed regime on both engines). The engine contrast is not a durability artifact: under vendor defaults it is, if anything, slightly larger than under the secondary, mechanism-isolating relaxed run. This also warrants caution: a write-throughput comparison run on a single engine invites the wrong generalization in either direction ("the access layer is irrelevant to writes" from MySQL alone, or "matters a great deal" from PostgreSQL alone), and an engine-only benchmark such as [12] cannot expose this interaction, since it omits the access layer entirely. Only the dual-engine design makes the engine-write-path explanation visible rather than confounded with layer choice, consistent with earlier quantitative ORM work showing database technology modulates mapping-strategy cost [29]; the ranking itself is engine-specific, since unlike every read pattern the write ranking does not transfer across engines (Spearman $\rho = 0.43$ versus $\rho \geq 0.89$ for reads). The single-row insert is not the only write shape: a transactional multi-statement write, inserting a post and five comments in one transaction, spreads the representative layers only 3.3× (Prisma 2412 down to MikroORM 740 req/s; Supplement Table S8), close to the single-row insert and far below the 6.55× read spread, because the transaction's single commit amortizes the per-commit flush across six inserts. The layer ordering tracks the single-row insert, so the write conclusions extend to that transactional pattern for the representative layers measured.

Methodological lessons: a checklist for access-layer benchmarking.. Reaching the figures above meant confronting four confounds that would otherwise have silently corrupted the comparison, and their provenance differs. One, the aggregation fan-out, is a *correctness* bug that our automated cross-check caught (the

same check, comparing each adapter's output against a native-driver reference on four of the five endpoints, also caught a driver result-shape mismatch); the other three return *correct* results and were therefore invisible to any equality check, found only by performance analysis. We regard the resulting checklist as reusable:

- **Aggregation fan-out.** A naive join between `posts` and `comments` evaluated before the `SUM` multiplies each post's view count by its number of matching comment rows, inflating the aggregate, a correctness bug invisible unless every layer's output is checked against a reference, not a performance one.
- **OFFSET vs. keyset pagination.** An early, OFFSET-based design for the range-scan endpoint collapsed on MySQL at realistic offsets, which would have been misread as an access-layer weakness rather than what it is: a pagination-strategy cost that InnoDB pays more dearly for than PostgreSQL. Rewriting every adapter around keyset pagination (`WHERE id < cursor ORDER BY id DESC LIMIT n`) removed the confound.
- **Write-workload contamination.** The insert endpoint grows `posts` on every call; because engines and layers commit at different rates, later cells in a run would scan a differently sized table depending on run order rather than on layer identity, unless benchmark-inserted rows are deleted before every cell.
- **Query-formulation confounds.** A first attempt at the aggregation endpoint pre-aggregated *all* of `comments` before filtering to one author, so every request re-scanned the whole table regardless of which author was requested, a query-plan artifact masquerading as an access-layer effect [66], fixed by rewriting the query as a correlated subquery touching only the target author's rows.

None of these four is schema-specific; each is a generic failure mode of the genre, extending the database-benchmarking pitfalls cataloged by Fruth et al. [7] and Raasveldt et al. [41] to the access-layer-comparison sub-genre. Each confound is easy to introduce by accident and hard to detect (the correctness ones without an automated cross-check, the performance ones without profiling every cell), and at least one plausibly explains part of the gap between some vendor-reported

and independently verified figures in this space.

Practical guidance.. The practical synthesis follows directly from RQ1–RQ3, on the default-configuration estimand a practitioner faces. Where a service's hot path is dominated by deep/nested fetches, the access layer is consequential, and a thin layer (the native driver, a query builder, or a query-engine-based ORM such as Prisma) is the safer default *for throughput and tail latency*: a heavy-weight data-mapper ORM concentrates its largest, growing penalty on that traffic, so weigh a query-engine ORM's parity or lead against its application-tier CPU cost wherever CPU, not throughput, is the constrained resource. Where the workload is read-simple (point reads, keyset scans) or write-bound, especially on MySQL, whose engine floor for single-row inserts dominates layer differences under default durability, a full ORM's productivity, type safety, and migration tooling cost little in relative throughput and are reasonable grounds for the decision on their own. A pragmatic middle path, already forced onto every layer by our aggregation endpoint, is to keep the ORM for the bulk of the application and fall back to raw SQL or the native driver only for endpoints that assemble large object graphs; the same-SQL control shows this fallback recovers most of the deep fetch's cost (Section 4), and, orthogonally, second-level caching can absorb part of an ORM's read cost where the workload tolerates staleness [27].

These are performance findings only: the access-layer decision also turns on developer productivity, type safety, correctness, maintainability, and security, none of which this study measures, and which may outweigh the throughput differences reported here. Even as performance guidance, the recommendations rest on one schema, one hardware configuration, a single-host local-database deployment measured under each engine's default durability for writes, and the pinned library versions; the same-SQL comparisons decompose that cost, not license to replace idiomatic defaults with raw SQL in practice. A production deployment adds network latency, TLS, a reverse proxy, a separate database host, connection poolers, read replicas, and larger or varied payloads, any of which can shift the balance; Section 6 sets out how far these results can generalize.

6. Threats to Validity

We organize the threats along the four standard validity categories [67, 51].

Construct validity.. Our dependent variables are HTTP-level throughput (req/s) and tail latency (p99) measured at the Express endpoint, not query-execution time at the driver or database boundary. This is deliberate, since it captures the full request round trip a deployed service incurs, including Express routing and JSON serialization, but it means the overhead we attribute to the *access layer* may partly reflect these surrounding costs instead. We control for this by holding those costs near-constant across conditions: every adapter satisfies a documented *adapter contract* (Section 3), and every endpoint's payload is now uniform by construction rather than only checked for equivalence: every adapter funnels its rows through shared canonical constructors (a fixed field set, key order, and value typing, under a fixed `TZ=UTC`), and an automated cross-check (`bench/verify.mjs`) asserted full JSON byte equality against the native-driver baseline, on both engines, across the four non-mutating patterns before any timing run. That check is itself evidence for the harness, not only a precondition of it: during development it caught a defect whereby PostgreSQL's schema declared `created_at` as a timezone-naive timestamp although the column is `timestamptz`, so Drizzle's PostgreSQL responses carried instants shifted by the host's UTC offset, a difference invisible to throughput and to any coarser equivalence check; it was fixed (`withTimezone: true`) before the campaign reported here. A fixed-payload baseline endpoint returning a fixed thread-shaped payload bounds the shared Express-plus-serialization floor well above the fastest measured cell (Section 3), so the framework floor compresses, rather than manufactures, between-layer differences. Observed differences between layers are therefore attributable to how each layer reaches an identical response, not to what is returned. Writes are the one endpoint that mutates state, and the reported finding uses each engine's *default*, vendor-standard durability configuration (Section 3), not a configuration we relaxed ourselves, so the reported write spreads (Table 7) and the roughly 7.5× native-driver

gap between engines (809 MySQL vs. 6,064 PostgreSQL req/s) describe a standard deployment rather than a tuning choice of ours. A labelled secondary run relaxes every durability setting on both engines symmetrically, including MySQL's binlog sync (an earlier check had left that one MySQL setting at its default, an asymmetry that did not affect the primary, default-durability result reported here but could have affected that secondary comparison specifically); under the symmetric relaxed regime throughput rises on both engines, but the cross-engine gap is, if anything, slightly larger than under defaults, so it is not an artifact of either durability configuration (Section 5, Supplement Table S3).

Internal validity.. A naively configured ORM can trigger N+1 queries and appear artificially slow for reasons unrelated to its inherent cost; every adapter must use its layer's documented, idiomatic eager-loading strategy on the deep-fetch endpoint (Section 3). The correctness cross-check has since been upgraded to the byte-level comparison described above (Construct validity); in its original, key-field-projection form it caught two defects during development, a fan-out bug that inflated the `SUM(views)` aggregate for every join-based adapter, and a result-shape parsing defect in one driver binding, both fixed before the measurements reported here. Explicit warm-up phases per (layer, engine, endpoint) cell, set to exceed the slowest layer's measured cold-start stabilization (Section 3), absorb JIT, pool-fill, and plan-cache effects before timing begins, guarding against environmentally induced measurement bias [68]; two further controls (Section 3) rule out non-layer confounds: benchmark-inserted rows are reset before every cell, from a physically rebuilt database state for writes, and the aggregation endpoint uses one correlated-subquery plan across layers, so it measures access-layer overhead rather than differences in SQL shape. Request target ids no longer risk contributing between-layer noise, since every layer and engine within a replicate is driven by the identical, PRNG-seeded id sequence (Section 3), and a forward-versus-reversed cell-order check found no detectable history effect, addressing the concern that a fixed processing order could confound layer identity with position in the run. A residual risk is coordinated omission in

the load generator, which can under-report tail latency at saturation [7, 59]; a native, coordinated-omission-corrected constant-arrival cross-run on the deep fetch (Section 3) confirms the tail ranking and shows the data-mapper ORMs saturating earlier still (Section 4; Supplement Table S6).

External validity.. All measurements come from a single schema, a single dataset size, one hardware configuration, and a connection pool fixed at ten, against specific engine and library versions (PostgreSQL 18.4, MySQL 9.7.1; Section 3). The fixed pool makes this an *equal-configuration* comparison, not a per-layer-tuned one: it isolates the access layer from pool tuning but does not show how fast each layer could be under its own best pool. A pool-size sensitivity sweep (Supplement Table S7) addresses this directly for a representative layer of each tier: the three-band ordering (native/query-engine ORM, then query builder, then data-mapper ORM) is preserved from a pool of 1 to 50, and each layer's throughput saturates well below 50, so the fixed-pool choice does not drive the ranking. Results may not transfer to larger working sets, other pool sizes, or other engine versions, and library performance ages quickly: Prisma's measured version (5.22) embeds the in-process Rust query engine our CPU accounting characterizes, whereas the version 7 line announced in late 2025 replaces it with a pure-TypeScript client [69], so the CPU findings describe the Rust-engine generation specifically, and re-running the harness against the new architecture is exactly the follow-up the instrument is built for. The host itself is a virtualized, 16-vCPU, single-NUMA-node cloud instance rather than dedicated bare metal, so absolute req/s figures reflect that substrate specifically; only the *relative* ordering within an engine is intended to travel, and we do not claim the absolute figures generalize beyond our hardware. Two further consequences of the design deserve emphasis: the working set is memory-resident, so these results are a hot-cache regime maximizing the visibility of layer differences; as a workload becomes I/O-bound (larger than RAM, cold caches), every layer increasingly waits on the same storage and the spreads should compress toward $1\times$. Second, the deep-fetch object graph is small (a post with on average ten comments, at

most 25 in the seed), so the headline spread is specific to small-graph hydration; the fan-out sweep (Section 4) shows the spread grows monotonically with graph size (to 16× at 500 child rows), so the headline figures are conservative in that direction. We mitigate these limitations by pinning every dependency version, publishing the deterministic seed and an automated environment capture, and releasing the full harness so the matrix can be re-run on other hardware or against newer releases. A further limitation is that the load generator and server under test share a host, common practice in this genre that removes network variance as a confound but can understate network-bound effects; process separation on one host is not hardware isolation, since the primary campaign did not pin the application, database, and generator to disjoint cores, so scheduler and shared-cache contention between them is possible (the environment capture records governor, NUMA, and affinity). A resource-isolation check re-ran the representative deep-fetch cells with the three components pinned to disjoint core sets (`ISO_` run, replication package): the isolated and shared-host medians agree within about 2% (ratios 0.98–1.01 across pg, Prisma, and MikroORM), and the layer ordering is unchanged, so same-host contention is not driving the reported ranking. The equal-CPU control (Section 4, Supplement Table S4) confirms that Prisma's near-native throughput depends on spare CPU: on an equal one-core budget the native driver reaches full throughput while Prisma is 4.1× slower, quantifying the subsidy rather than inferring it from aggregate CPU%. The same caveat extends to deployment topology: the server runs as a single Node.js process, whereas production deployments typically run one worker per core (`cluster`, pm2, or per-pod replicas); under N workers saturating all cores, no spare cores remain for Prisma's engine threads, and the single-threaded layers' per-core efficiency should dominate, so the CPU-for-latency trade we report is a single-process result. A sustained-load check bounds temporal drift: holding three representative layers (native `pg`, Prisma, MikroORM) under ten minutes of continuous deep-fetch load, re-run under the final harness, moves throughput by at most ±1.4% between the first and last three minutes, with the band ordering preserved in every single minute (replication package), so the short measured

runs are not sampling a transient; longer-horizon drift over hours to days [4] remains untested. A two-machine cross-run over a dedicated link, adding real network latency, and a multi-worker variant are planned to bound this further.

Conclusion validity.. Our analysis (Section 3) reports medians with the coefficient of variation and bootstrap 95% confidence intervals as stability signals and, because the design is a randomized block, compares adjacently ranked layers with *paired* tests on their per-replicate throughput ratios (paired sign-flip permutation, cross-checked with the Wilcoxon signed-rank test; Table 9). Two bounds reinforce the ordering (Section 4): the widest spreads dwarf the run-to-run CV (at most 7.6% on PostgreSQL and 6.6% on MySQL; Supplement Tables S9 and S1), and the three-band ordering holds across the concurrency sweep at ≥ 32 connections (Fig. 1). On the deep/nested fetch, six of the seven adjacent pairs separate at the permutation-resolution floor with the faster layer ahead in nearly every replicate, each surviving a Bonferroni correction across the confirmatory family (Table 9); the exception, native `pg` and Prisma at the top, is not resolved as different by the paired test and is confirmed equivalent by a paired TOST procedure within an a-priori $\pm 5\%$ margin, so we read those two as tied rather than ranked. Rather than assert a specific statistical power, which would require a prospective effect-size and variance model we did not prespecify, we rest the conclusions on effect sizes and interval estimates: the separating pairs combine large paired ratios with the permutation floor and non-overlapping bootstrap intervals, and the inference is carried jointly by that separation and by spreads well above the run-to-run dispersion. Booting a fresh process per replicate prevents persistent process state from carrying across replicate runs [70], but the replicates share one host and one campaign; we therefore treat replicate index and measurement order as blocks (Section 3) rather than claiming full independence, and more replicates or longer runs would tighten the intervals further.

7. Conclusion and Future Work

This paper set out to replace vendor-benchmark claims with a vendor-independent, reproducible comparison of relational database access layers in Express.js, serving an identical application through a representative taxonomy (native drivers, a query builder, and six ORMs) across PostgreSQL and MySQL, over five access patterns, and reporting throughput and tail latency together, addressing the four gaps that characterize prior art (Section 1).

The empirical answer is that the cost of abstraction is concentrated, not uniform. It is modest for point reads and for aggregation issued through a raw-SQL path, but grows sharply for the deep/nested fetch, where the layer must assemble a nested object graph on the application side, and widens further as the fetched graph grows larger (Section 4). A same-SQL control shows that most of this cost lies upstream of raw SQL execution, in each layer's default query-generation and result-construction path rather than in its execution engine, with MikroORM the starkest case (Section 4). Read rankings are stable across the two engines, and a competitively fast modern ORM, Prisma, shows that "ORM" need not imply large overhead: on the deep fetch it is formally equivalent to the native driver and leads outright on several other patterns, at a substantial application-CPU premium that an equal-CPU control now measures directly rather than infers (Section 4). Writes are the exception: MySQL's engine floor makes insert throughput nearly layer-independent, while PostgreSQL exposes a wide layer spread even under default durability, and unlike every read pattern the write ranking does not transfer across engines at all, a caution against generalizing single-engine write results, not a durability artifact as a symmetric relaxed-durability run confirms (Section 5). Development also surfaced a reusable checklist of benchmarking pitfalls (aggregation fan-out, `OFFSET`-versus-keyset pagination, write-workload contamination, and query-formulation confounds), several caught by the correctness cross-check and the rest by performance analysis (Section 5).

These findings held under a concurrency sweep (1 to 256 connections, ranking

load-robust above 32) and under paired permutation and Wilcoxon tests confirming adjacent-layer separation (Section 4). Further extensions would strengthen the evidence: a two-machine cross-run isolating the load generator would bound observer-interference effects beyond the open-loop check already performed here; additional engines and dataset sizes would widen external validity; and a dedicated study of the N+1 penalty and of cold-start latency would deepen the practical guidance. Beyond these specific findings, the durable contribution is the harness itself: a vendor-independent, dual-engine instrument with a correctness cross-check and a same-SQL decomposition control, released under open licenses so that these extensions, and independent re-runs against current library versions, are straightforward, addressing the repeatability gap widely documented in computer-systems research [45].

CRediT authorship contribution statement

Mateusz Miotk: Conceptualization, Methodology, Software, Investigation, Data curation, Formal analysis, Visualization, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The author declares that he has no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper. The author is not affiliated with, nor funded by, any of the database libraries or vendors evaluated in this study.

Data availability

The complete replication package (benchmark harness, database schema and deterministic seed, all adapter implementations, the byte-level correctness cross-check, raw per-cell measurements, and the scripts that generate every table in this paper) is publicly available at `https://github.com/mmiotk/`

express-db-access-performance and permanently archived at Zenodo as release v1.2.0 (https://doi.org/10.5281/zenodo.21356266).

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the author used a large language model (Claude, Anthropic) to assist with drafting and editing the manuscript and with implementing and analyzing the benchmark harness. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the published article.

References

[1] T.-H. Chen, W. Shang, Z. M. Jiang, A. E. Hassan, M. Nasser, P. Flora, Detecting performance anti-patterns for applications developed using object-relational mapping, in: Proceedings of the 36th International Conference on Software Engineering (ICSE), ACM, 2014, pp. 1001–1012. doi:10.1145/2568225.2568259.

[2] M. Fowler, Patterns of Enterprise Application Architecture, Addison-Wesley, Boston, MA, 2002.

[3] I. K. Chaniotis, K.-I. D. Kyriakou, N. D. Tselikas, Is Node.js a viable option for building modern web applications? a performance evaluation study, Computing 97 (10) (2015) 1023–1044. doi:10.1007/s00607-014-0394-9.

[4] P. Kuffeł, B. Walter, Changes in Node.js server-side application performance characteristics over time under load, in: Advances in Information

Systems Development, Vol. 77 of Lecture Notes in Information Systems and Organisation, Springer, 2025, pp. 275–294. doi:10.1007/978-3-031-87880-0_14.

[5] K. X. Carmo, F. Ferreira, E. Figueiredo, Performance evaluation of back-end frameworks: A comparative study, in: Proceedings of the 20th Brazilian Symposium on Information Systems (SBSI), ACM, 2024, pp. 1–9. doi:10.1145/3658271.3658314.

[6] J. Dean, L. A. Barroso, The tail at scale, Commun. ACM 56 (2) (2013) 74–80. doi:10.1145/2408776.2408794.

[7] M. Fruth, S. Scherzinger, W. Mauerer, R. Ramsauer, Tell-tale tail latencies: Pitfalls and perils in database benchmarking, in: Performance Evaluation and Benchmarking (TPCTC 2021), Vol. 13169 of Lecture Notes in Computer Science, Springer, 2022, pp. 119–134. doi:10.1007/978-3-030-94437-7_8.

[8] D. Colley, C. Stanier, M. Asaduzzaman, The impact of object-relational mapping frameworks on relational query performance, in: 2018 International Conference on Computing, Electronics & Communications Engineering (iCCECE), IEEE, 2018, pp. 47–52, java, C#, Python, PHP — deliberately excludes JavaScript/Node. doi:10.1109/iccecome.2018.8659222.

[9] J. C. Yusmita, R. Arya, J. M. Wijaya, K. M. Suryaningrum, R. R. Siswanto, Optimizing database access strategy: A performance analysis comparison of raw sql and prisma orm, Procedia Comput. Sci. 269 (2025) 1201–1210. doi:10.1016/j.procs.2025.09.061.

[10] J. Attala, C. Khemapatpapan, Comparing the performance of prisma, typeorm, and sequelize on postgresql, J. Comput. Creat. Technol. 3 (2) (2025) 201–213. doi:10.14456/jcct.2025.16.
URL https://so13.tci-thaijo.org/index.php/jcct/article/view/2330

[11] S. Zhadko-Bazilevych, Analysis of ORM framework approaches for Node.js, J. Comput. Sci. Inst. 37 (2025) 426–430. doi:10.35784/jcsi.7951.

[12] S. V. Salunke, A. Ouda, A performance benchmark for the PostgreSQL and MySQL databases, Future Internet 16 (10) (2024) 382. doi:10.3390/fi16100382.

[13] Drizzle Team, Drizzle orm benchmarks, `https://orm.drizzle.team/benchmarks`, k6, 1M prepared requests, two-machine setup; reports req/s, avg + P95 latency, CPU (accessed 10 July 2026). (2024).

[14] M. D. Imam Sakti, D. Dirgantara, A. K. Ramadhan, M. A. Ibrahim, Optimizing asynchronous performance in Node.js with Express and PostgreSQL, Procedia Comput. Sci. 269 (2025) 172–181. doi:10.1016/j.procs.2025.08.270.

[15] M. Collina, et al., autocannon: fast http/1.1 benchmarking tool for node.js, `https://github.com/mcollina/autocannon`, version 7.15.0; accessed 10 July 2026. (2023).

[16] C. Ireland, D. Bowers, M. Newton, K. Waugh, A classification of object-relational impedance mismatch, in: 2009 First International Conference on Advances in Databases, Knowledge, and Data Applications (DBKDA), IEEE, 2009, pp. 36–43. doi:10.1109/DBKDA.2009.11.

[17] C. Russell, Bridging the object-relational divide, ACM Queue 6 (3) (2008) 18–28. doi:10.1145/1394127.1394139.

[18] A. Torres, R. Galante, M. S. Pimenta, A. J. B. Martins, Twenty years of object-relational mapping: A survey on patterns, solutions, and their implications on application design, Inf. Softw. Technol. 82 (2017) 1–18. doi:10.1016/j.infsof.2016.09.009.

[19] P. J. Sadalage, M. Fowler, NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence, Addison-Wesley, Upper Saddle River, NJ, 2012.

[20] J. Yang, P. Subramaniam, S. Lu, C. Yan, A. Cheung, How not to structure your database-backed web applications: A study of performance bugs in the wild, in: Proceedings of the 40th International Conference on Software Engineering (ICSE), ACM, 2018, pp. 800–810. doi:10.1145/3180155.3180194.

[21] C. Yan, A. Cheung, J. Yang, S. Lu, Understanding database performance inefficiencies in real-world web applications, in: Proceedings of the 2017 ACM Conference on Information and Knowledge Management (CIKM), ACM, 2017, pp. 1299–1308. doi:10.1145/3132847.3132954.

[22] S. Shao, Z. Qiu, X. Yu, W. Yang, G. Jin, T. Xie, X. Wu, Database-access performance antipatterns in database-backed web applications, in: 2020 IEEE International Conference on Software Maintenance and Evolution (ICSME), IEEE, 2020, pp. 58–69. doi:10.1109/ICSME46990.2020.00016.

[23] T.-H. Chen, W. Shang, Z. M. Jiang, A. E. Hassan, M. Nasser, P. Flora, Finding and evaluating the performance impact of redundant data access for applications that are developed using object-relational mapping frameworks, IEEE Trans. Softw. Eng. 42 (12) (2016) 1148–1161. doi:10.1109/TSE.2016.2553039.

[24] A. Cheung, A. Solar-Lezama, S. Madden, Optimizing database-backed applications with query synthesis, in: Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), ACM, 2013, pp. 3–14. doi:10.1145/2491956.2462180.

[25] A. Cheung, S. Madden, A. Solar-Lezama, Sloth: Being lazy is a virtue (when issuing database queries), in: Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data, ACM, 2014, pp. 931–942. doi:10.1145/2588555.2593672.

[26] J. Yang, C. Yan, C. Wan, S. Lu, A. Cheung, View-centric performance optimization for database-backed web applications, in: Proceedings of the 41st International Conference on Software Engineering (ICSE), IEEE, 2019, pp. 994–1004. doi:10.1109/ICSE.2019.00104.

[27] T.-H. Chen, W. Shang, A. E. Hassan, M. Nasser, P. Flora, CacheOptimizer: Helping developers configure caching frameworks for Hibernate-based database-centric web applications, in: Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE), ACM, 2016, pp. 666–677. doi:10.1145/2950290.2950303.

[28] A. Turcotte, M. D. Shah, M. W. Aldrich, F. Tip, DrAsync: Identifying and visualizing anti-patterns in asynchronous JavaScript, in: Proceedings of the 44th International Conference on Software Engineering (ICSE), ACM, 2022, pp. 774–785. doi:10.1145/3510003.3510097.

[29] M. Lorenz, J.-P. Rudolph, G. Hesse, M. Uflacker, H. Plattner, Object-relational mapping revisited—a quantitative study on the impact of database technology on O/R mapping strategies, in: Proceedings of the 50th Hawaii International Conference on System Sciences (HICSS), 2017. doi:10.24251/HICSS.2017.592.

[30] P. van Zyl, D. G. Kourie, L. Coetzee, A. Boake, The influence of optimisations on the performance of an object relational mapping tool, in: Proceedings of the 2009 Annual Research Conference of the South African Institute of Computer Scientists and Information Technologists (SAICSIT), ACM, 2009, pp. 150–159. doi:10.1145/1632149.1632169.

[31] A. Bonvoisin, C. Quinton, R. Rouvoy, Understanding the performance-energy tradeoffs of object-relational mapping frameworks, in: 2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), IEEE, 2024, pp. 626–636. doi:10.1109/SANER60148.2024.00069.

[32] P. van Zyl, D. G. Kourie, A. Boake, Comparing the performance of object databases and ORM tools, in: Proceedings of the 2006 Annual Research Conference of the South African Institute of Computer Scientists and Information Technologists (SAICSIT), ACM, 2006, pp. 1–11. doi:10.1145/1216262.1216263.

[33] Y. Li, S. Manoharan, A performance comparison of SQL and NoSQL databases, in: 2013 IEEE Pacific Rim Conference on Communications, Computers and Signal Processing (PACRIM), IEEE, 2013, pp. 15–19. doi:10.1109/PACRIM.2013.6625441.

[34] C. Gyorodi, R. Gyorodi, G. Pecherle, A. Olah, A comparative study: MongoDB vs. MySQL, in: 2015 13th International Conference on Engineering of Modern Electric Systems (EMES), IEEE, 2015, pp. 1–6. doi:10.1109/EMES.2015.7158433.

[35] R. Cattell, Scalable SQL and NoSQL data stores, ACM SIGMOD Rec. 39 (4) (2011) 12–27. doi:10.1145/1978915.1978919.

[36] F. Gessert, W. Wingerath, S. Friedrich, N. Ritter, NoSQL database systems: A survey and decision guidance, Comput. Sci. Res. Dev. 32 (3–4) (2017) 353–365. doi:10.1007/s00450-016-0334-3.

[37] H. Sun, D. Bonetta, F. Schiavio, W. Binder, Reasoning about the Node.js event loop using async graphs, in: 2019 IEEE/ACM International Symposium on Code Generation and Optimization (CGO), IEEE, 2019, pp. 61–72. doi:10.1109/CGO.2019.8661173.

[38] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, R. Sears, Benchmarking cloud serving systems with YCSB, in: Proceedings of the 1st ACM Symposium on Cloud Computing (SoCC), ACM, 2010, pp. 143–154. doi:10.1145/1807128.1807152.

[39] S. Chen, A. Ailamaki, M. Athanassoulis, P. B. Gibbons, R. Johnson, I. Pandis, R. Stoica, TPC-E vs. TPC-C: Characterizing the new TPC-E benchmark via an I/O comparison study, ACM SIGMOD Rec. 39 (3) (2011) 5–10. doi:10.1145/1942776.1942778.

[40] D. E. Difallah, A. Pavlo, C. Curino, P. Cudré-Mauroux, OLTP-Bench: An extensible testbed for benchmarking relational databases, Proc. VLDB Endow. 7 (4) (2013) 277–288. doi:10.14778/2732240.2732246.

[41] M. Raasveldt, P. Holanda, T. Gubner, H. Mühleisen, Fair benchmarking considered difficult: Common pitfalls in database performance testing, in: Proceedings of the Workshop on Testing Database Systems (DBTest), ACM, 2018, pp. 1–6. doi:10.1145/3209950.3209955.

[42] J. Li, N. K. Sharma, D. R. K. Ports, S. D. Gribble, Tales of the tail: Hardware, OS, and application-level sources of tail latency, in: Proceedings of the ACM Symposium on Cloud Computing (SoCC), ACM, 2014, pp. 1–14. doi:10.1145/2670979.2670988.

[43] Z. M. Jiang, A. E. Hassan, A survey on load testing of large-scale software systems, IEEE Trans. Softw. Eng. 41 (11) (2015) 1091–1118. doi:10.1109/TSE.2015.2445340.

[44] P. Leitner, C.-P. Bezemer, An exploratory study of the state of practice of performance testing in Java-based open source projects, in: Proceedings of the 8th ACM/SPEC International Conference on Performance Engineering (ICPE), ACM, 2017, pp. 373–384. doi:10.1145/3030207.3030213.

[45] C. Collberg, T. A. Proebsting, Repeatability in computer systems research, Commun. ACM 59 (3) (2016) 62–69. doi:10.1145/2812803.

[46] K. Huppler, The art of building a good benchmark, in: Performance Evaluation and Benchmarking (TPCTC 2009), Vol. 5895 of Lecture Notes in Computer Science, Springer, 2009, pp. 18–30. doi:10.1007/978-3-642-10424-4_3.

[47] S. E. Sim, S. Easterbrook, R. C. Holt, Using benchmarking to advance research: A challenge to software engineering, in: Proceedings of the 25th International Conference on Software Engineering (ICSE), IEEE, 2003, pp. 74–83. doi:10.1109/ICSE.2003.1201189.

[48] Prisma, Query performance benchmarks, `https://benchmarks.prisma.io/`, prisma/Drizzle/TypeORM; median of 500 iterations; no throughput, no p95/p99 (accessed 10 July 2026). (2024).

[49] Gel Data (EdgeDB), Imdbench: Orm benchmarks, `https://github.com/geldata/imdbench`, accessed 10 July 2026. (2024).

[50] V. R. Basili, H. D. Rombach, The TAME project: Towards improvement-oriented software environments, IEEE Trans. Softw. Eng. 14 (6) (1988) 758–773. doi:10.1109/32.6156.

[51] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, A. Wesslén, Experimentation in Software Engineering, 2nd Edition, Springer, Berlin, Heidelberg, 2012. doi:10.1007/978-3-642-29044-2.

[52] B. A. Kitchenham, S. L. Pfleeger, L. M. Pickard, P. W. Jones, D. C. Hoaglin, K. El Emam, J. Rosenberg, Preliminary guidelines for empirical research in software engineering, IEEE Trans. Softw. Eng. 28 (8) (2002) 721–734. doi:10.1109/TSE.2002.1027796.

[53] S. Harizopoulos, D. J. Abadi, S. Madden, M. Stonebraker, OLTP through the looking glass, and what we found there, in: Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data, ACM, 2008, pp. 981–992. doi:10.1145/1376616.1376713.

[54] K. Gupta, M. Mathuria, Improving performance of web application approaches using connection pooling, in: 2017 International Conference of Electronics, Communication and Aerospace Technology (ICECA), IEEE, 2017, pp. 355–358. doi:10.1109/ICECA.2017.8212833.

[55] R. Laigner, Y. Zhou, M. A. Vaz Salles, Y. Liu, M. Kalinowski, Data management in microservices: State of the practice, challenges, and research directions, Proc. VLDB Endow. 14 (13) (2021) 3348–3361. doi:10.14778/3484224.3484232.

[56] B. Schroeder, A. Wierman, M. Harchol-Balter, Open versus closed: A cautionary tale, in: Proceedings of the 3rd USENIX Symposium on Networked Systems Design and Implementation (NSDI), USENIX Association, 2006.

URL https://www.usenix.org/legacy/event/nsdi06/tech/schroeder.html

[57] E. Barrett, C. F. Bolz-Tereick, R. Killick, S. Mount, L. Tratt, Virtual machine warmup blows hot and cold, Proceedings of the ACM on Programming Languages 1 (OOPSLA) (2017) 1–27. doi:10.1145/3133876.

[58] X. Yu, G. Bezerra, A. Pavlo, S. Devadas, M. Stonebraker, Staring into the abyss: An evaluation of concurrency control with one thousand cores, Proc. VLDB Endow. 8 (3) (2014) 209–220. doi:10.14778/2735508.2735511.

[59] G. Tene, How NOT to measure latency, https://www.infoq.com/presentations/latency-response-time/, conference presentation; origin of the term "coordinated omission" (accessed 10 July 2026). (2015).

[60] B. Krishnamachari, How to do statistical evaluations in ECE/CS papers: A practical playbook for defensible results, https://arxiv.org/abs/2605.00428 (2026).

[61] A. Georges, D. Buytaert, L. Eeckhout, Statistically rigorous Java performance evaluation, in: Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA), ACM, 2007, pp. 57–76. doi:10.1145/1297027.1297033.

[62] C. Laaber, J. Scheuner, P. Leitner, Software microbenchmarking in the cloud. how bad is it really?, Empir. Softw. Eng. 24 (4) (2019) 2469–2508. doi:10.1007/s10664-019-09681-1.

[63] A. Arcuri, L. Briand, A hitchhiker's guide to statistical tests for assessing randomized algorithms in software engineering, Softw. Test. Verif. Reliab. 24 (3) (2014) 219–250. doi:10.1002/stvr.1486.

[64] N. Cliff, Dominance statistics: Ordinal analyses to answer ordinal questions, Psychol. Bull. 114 (3) (1993) 494–509. doi:10.1037/0033-2909.114.3.494.

[65] L. M. Vaquero, L. Rodero-Merino, R. Buyya, Dynamically scaling applications in the cloud, ACM SIGCOMM Comput. Commun. Rev. 41 (1) (2011) 45–52. doi:10.1145/1925861.1925869.

[66] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, T. Neumann, How good are query optimizers, really?, Proc. VLDB Endow. 9 (3) (2015) 204–215. doi:10.14778/2850583.2850594.

[67] T. D. Cook, D. T. Campbell, Quasi-Experimentation: Design and Analysis Issues for Field Settings, Houghton Mifflin, Boston, MA, 1979.

[68] T. Mytkowicz, A. Diwan, M. Hauswirth, P. F. Sweeney, Producing wrong data without doing anything obviously wrong!, in: Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), ACM, 2009, pp. 265–276. doi:10.1145/1508244.1508275.

[69] Prisma, Prisma 7 performance and benchmarks, `https://www.prisma.io/blog/prisma-7-performance-benchmarks`, announces the query-engine rewrite from the Rust engine to a TypeScript client (accessed 13 July 2026). (2025).

[70] T. Kalibera, R. Jones, Rigorous benchmarking in reasonable time, in: Proceedings of the 2013 International Symposium on Memory Management (ISMM), ACM, 2013, pp. 63–74. doi:10.1145/2464157.2464160.